

Dear Meta Reviewer and Referees:

We would like to thank the Referees for insightful comments and for supporting this work! We have substantially revised the paper following the Referees' suggestions.

For the convenience of Referees, changes for addressing the concerns of R1, R2, R3 and R4 are color-coded in blue, red, orange and purple, respectively. The source code and data are available in [1], and the full version is [2], providing more algorithm details and additional experimental results.

Below please find our responses to the reviews.

Response to the comments of the Meta Reviewer.

[MetaO1] *However, despite these strengths, reviewers raise concerns regarding clarity, justification of design choices, and reproducibility, particularly for components that rely on heuristic decisions and LLM-based predicate ordering.*

- *The clarity and structure can be improved, since the paper is at places overly dense, for example, better motivate and explain in context the role of 1-WL predicates and provide a high-level view of how the major components fit together.*
- *The paper needs stronger transparency and empirical justification for key design choices. The LLM-based predicate ordering component must be described in detail and its impact quantified. Additional ablation studies are asked to isolate the contribution of major components (e.g., removing LLM-based ordering or 1-WL predicates) and to assess sensitivity to heuristic parameters.*

[A] Thanks! We have substantially revised the paper and addressed comments of all referees. More specifically, we have:

- Improved clarity in the response to R1[O1, O2, O3], R3[O1, Additional remarks] and R4[O3, Availability], by adding the intuition of 1-WL test, providing a workflow of NEX, revising the figures to visualize the inter-connection in NEX, clarifying how often the offline rule discovery is conducted, discussing the extension beyond binary GNNs, improving the paddings of Tables III-IV, explaining the behavior on Loan, clarifying the use of support and confidence and providing more details of LLMs in our artifacts.
- Revised the presentation in the response to R1[Additional remarks], R2[O1] and R3[Additional remarks], by revising the figures, providing more intuitions and justifications, and re-organizing the techniques to facilitate understanding.
- Justified the design choices in the response to R1[O5] and R4[O1, O2, Q12], by adding justification for major components (e.g., DQN and predicate ordering) and clarifying the LLM-based predicate ordering as a speedup optimization.
- Expanded experiments in the response to R1[O4, O5] and R4[O1, O2, Q12], by substantially re-organizing the ablation study to isolate the contribution of major components (in particular, the predicate ordering and 1-WL predicates) and providing supplementary case studies in [2].

Response to the comments of Referee #1.

[R1O1] *The 1-WL test has been defined well in the preliminaries, but its use in the context of the problem was not very clear to me. It would help if we could include this information either in Example 4 or right after it.*

[A] Thanks! As suggested, we have added the intuition about 1-WL test right after Example 4 (pp. 3). Intuitively, the 1-WL test is a classical graph-theoretic technique widely used for comparing structural similarity between vertices. If two vertices x and y fall into the same 1-WL equivalence class, they share highly similar topological contexts. Since most existing GNN architectures are known to be no more expressive than the 1-WL test [3], [4], [5], [6], if $\mathcal{M}$ predicts a negative outcome at y , it is likely to behave similarly at x . In this sense, using the 1-WL test as a predicate equips our rules with sufficient expressive capability to characterize and explain the behavior patterns of GNN models. Example RxGNNs presented in Example 5 (pp. 2, 4) also provide the intuitive meanings of 1-WL test under different concrete contexts.

[R1O2] *The paper has many concepts (RxGNN semantics, motif mining, DQN based composition, LLM based predicate ordering, candidate spaces, cost effectiveness, clustering etc.) that make it dense and easy to lose track of how all components fit together.*

[A] Thanks! We have added a workflow of NEX in Section III (pp. 5) that describes how the components of NEX work collaboratively to generate global and local explanations for negative GNNs. Besides, we have enriched Figure 3 (pp. 4) to visualize the inter-connection among these components.

For global explanations, since RxGNNs consist of patterns Q and preconditions X , *DQN-guided composition* and *predicate ordering* are designed to accelerate pattern generation and precondition generation in rule discovery, respectively.

For local explanations, we represent the search space concisely by a *candidate space*, and compute the *cost-effectiveness* of perturbations with *clustering* to get the right one to apply.

[R1O3] *It is not entirely clear how often the offline rule discovery procedure would need to be re-run for new models or for updated data.*

[A] As suggested, we have added a remark in Section IV (pp. 7) to explain how often the offline rule discovery procedure is conducted and when it is necessary to do so.

In practice, data is often continually updated and model's predictions on certain part of the data may change when the model evolves. To maintain the effectiveness of the rules in the set Σ under such dynamic conditions, one can use an indicator that periodically samples a portion of the dataset to validate the rules based on their support and confidence. If the indicator detects a decline in the effectiveness of the rules (e.g., if their support or confidence falls below a predetermined threshold on the updated data), we re-run the offline rule discovery procedure. Alternatively, one can also use incremental discovery in response to frequent yet small updates.

[R1O4] *The paper does not clearly quantify how much the*

LLM-based perplexity for predicate ordering actually contributes to the final performance.

[A] We respectfully clarify that in our original submission, we have conducted an ablation study by comparing a variant of NEX without the LLM-based perplexity predicate ordering (PPO). Instead, this variant processes predicates based on the descending order of their supports following [7]. The use of PPO speeds up rule discovery by $1.21\times$ (now reported in Figure 10(b), and pp. 12), since PPO optimizes multi-thread parallelism. Please also see our responses to [R105] and [R402].

[R105] *The paper misses ablation studies where 1-WL predicates are disabled or LLM-based ordering is removed while keeping the rest of NEX intact. Please clarify the need for LLM-based components further and quantify their contribution in terms of both efficiency and explanation quality. This helps to better assess the effect on quality of rules and explanations.*

[A] Thanks! As suggested, we have modified the following.

(1) We have further justified the needs for advanced predicate ordering schemes in Section IV (pp. 7).

Recall that we use multiple threads for precondition generation, where each thread *independently* handles a task $\langle X, \mathcal{P} \rangle$. To illustrate, consider two tasks $t_1 = \langle X_1, \mathcal{P}_1 \rangle$ and $t_2 = \langle X_2, \mathcal{P}_2 \rangle$ for Q , where $X_1 = \{p_1\}$ and $X_2 = \{p_2\}$. Denote the RxGNNs for t_1 and t_2 by ϕ_1 and ϕ_2 , respectively, and assume that ϕ_1 is σ -frequent but ϕ_2 is not. If t_1 is processed before t_2 , we may expand $X_1 = \{p_1\}$ to new $X_{1,2} = \{p_1, p_2\}$, yielding a new task $t_{1,2} = \langle X_{1,2}, \mathcal{P}_{1,2} \rangle$. However, we will not generate this task if we process t_2 first, since ϕ_2 is not σ -frequent. This motivates us to develop an effective predicate ordering so that *discriminative* predicates are expanded early.

(2) We have clarified PPO is a *speedup optimization* which does not affect rule quality, *i.e.*, it will find exactly the same set of preconditions, no matter what ordering is used. In other words, the interpretability of our framework remains intact, with or without this optimization. Nevertheless, the use of PPO can process discriminative predicates first, so that the pruning based on anti-monotonicity (pp. 5) can be applied earlier.

Continuing with the example above, although a new task $t_{1,2} = \langle X_{1,2}, \mathcal{P}_{1,2} \rangle$ is generated if we process t_1 first, it only incurs additional processing time and after processing $t_{1,2}$, we will find no new valid rule. To see why this is the case, consider the rule $\phi_{1,2} = Q[\bar{x}, x_0](X_{1,2} \rightarrow \neg \mathcal{M}(x_0))$ for $t_{1,2}$. Note that $\phi_{1,2}$ is subsumed by ϕ_2 , which is known to be not σ -frequent. According to the anti-monotonicity property, $\phi_{1,2}$ is also not σ -frequent and thus, will eventually be pruned. However, this information is not utilized if we process t_1 first and we can prune $\phi_{1,2}$ only after actual (but unnecessary) validation.

(3) We have re-organized ablation studies by disabling/varying each component while keeping the rest of NEX intact (pp. 12).

In particular, we have expanded the experiment for predicate orderings to a standalone test (Figure 10(b) and pp. 12), by replacing PPO with (a) a support-based order (SO) and (b) a random order (RO). Here all orders only differ in efficiency and

they produce exactly the same rules. PPO is $1.21\times$ and $1.44\times$ faster than SO and RO, respectively, since it improves multi-thread parallelism by prioritizing discriminative predicates. SO is even slower than RO on Claim since processing high-support predicates may delay pruning (see [2] for a case study).

(4) We have added an ablation study for 1-WL predicates in Section VI (pp. 12). More specifically, the expressivity of RxGNNs without 1-WL predicates is greatly reduced, *e.g.*, on Amazon, recog of NEX falls from 0.731 to 0.537 and relia from 0.604 to 0.417, justifying the need of 1-WL for GNNs.

[R1 Additional remarks] *Some figures are conceptually helpful, but difficult to map to the latter generic algorithm.*

[A] Thanks! We have enriched Figure 3 (pp. 4) to visualize the inter-connection of its components. Moreover, we have also improved Figures 1 and 5 (pp. 1, 6) for better readability.

Many thanks for noting the need and benefit of NEX!

Response to the comments of Referee #2.

[R201] *The writing is very dense and notation is heavy at times. Despite the many examples, explanations are sometimes a bit terse.*

[A] Thanks! We have substantially improved the presentation by (1) adding the workflow of NEX in Section III (pp. 3) to illustrate high-level interactions, (2) improving Figures 1, 3 and 5 (pp. 1, 4, 6) for better readability, (3) providing more intuitions for critical techniques (*e.g.*, the justification for using DQN for motif composition (pp. 6) and an example for the need of advanced predicate ordering (pp. 7), etc), (4) adding explanations for examples (*e.g.*, Example 11, pp. 8).

Besides, we have re-organized the techniques to facilitate understanding, *e.g.*, we have (1) moved the details of feature modification (pp. 9) to the online full version [2] so that the notation is simplified; (2) simplified the description of RxGNNMiner (pp. 7) by encapsulating pattern generation; and (3) re-structured Section V-A (pp. 8) by downplaying the candidate set $C_Q(x)$ for matching pattern Q and to focus directly on the candidate space C_φ , which is the main emphasis.

Many thanks for your support! We truly appreciate it!

Response to the comments of Referee #3.

[R301] *While the paper studies an already difficult problem, yet, it would also be interesting to discuss if/how this approach could be extended to go beyond binary GNN classifiers.*

[A] Thanks! In our original submission, we have discussed an extension of NEX to multi-class GNN classifiers (now on pp. 5). Specifically, $\neg \mathcal{M}(x_0)$ in RxGNNs can be replaced with an ML predicate $\mathcal{M}(x_0) = i$, where $i \in [1, K]$ denotes the class predicted for x_0 by $\mathcal{M}$ (note that $K = 2$ recovers binary classification). In this setting, counterfactual explanations aim to change the predicted class of x_0 from i to another class. We have tested this extension in Section VI by employing a multi-class e-commerce graph Amazon [8] (Tables III-IV).

Extending NEX to models beyond GNNs is an important direction, which we leave for future work (pp. 12). The approach is promising because ML models without chain-of-thought reasoning are generally considered less expressive than first-order logic, even though the precise logical expressiveness of most models remains uncharacterized. This suggests that logical explanations may also be obtainable for other models.

[R3 Additional remarks] *Table 2 and table 3 look inconsistent at the paddings. The whole page seems too dense. Explain why recog is 1 in the loan dataset on Table IV. I don't think that an acknowledgment is needed for NOT using GenAI.*

[A] Thanks! As suggested, we have revised the paddings of Tables III-IV to make them consistent. Moreover, we have explained why recog is 1 on Loan (pp. 11). This is because the prediction of loan application is highly individual-related, exhibiting clear regional characteristic (e.g., low salary or lack of experience). All GNNs have learned these suspicious features, which also appear in nearly all RxGNNs.

Although our content (text, figures, images and code) are not generated by AI, according to the submission guideline of ICDE, all submissions must include an "AI-Generated Content Acknowledgment" section, placed before the References.

Many thanks for your support! We sincerely appreciate it!

Response to the comments of Referee #4.

[R401 & Q12] *Their system is simple and numerous heuristics are employed in order for their methodology to work. For instance, using frequent subgraph mining to find motifs, using a deep Q-learning policy to merge important motifs, predicate generation via significance testing, confidence/support pruning to name a few. These methods are too dependent on the heuristics that are employed and can dramatically alter the outputs if hyperparameters and design choices are not carefully selected.*

There are several components incorporated into their framework that can be very sensitive and result in vastly different overall outcomes when minimally modified. For these reasons extensive justification is needed on how these independent components can come together to generate reasonable solutions for their problem. Also they employ GASTON for frequent subgraph mining and cite a link to their own paper stating it is empirical evidence of superiority of GASTON. For all independent components included extensive evidence of their reliability, and efficacy should be provided.

[A] Thanks! Several components work collaboratively in our framework to generate explanations for negative GNN predictions. While some of these techniques involve heuristic, they are designed to either (a) enhance efficiency without compromising accuracy (e.g., the predicate ordering as a speedup optimization, verified by the experiments), or (b) balance exactness and scalability (e.g., DQN to avoid exhaustive search), or (c) adhere to standard practices in relevant literature (e.g., restricted predicate space to reduce cost), as elaborated below.

(1) *Pattern mining.* Different techniques can be used to im-

plement the primitive generator for pattern mining. We have moved the experiment that tests the time and results of various mining methods from [2] to the main text (Section VI, pp. 12), where we compared GASTON [9] (our default) with other methods, including gSpan [10], FSG [11] and MoFa [12].

Since all methods run exact mining given same inputs and mining requirements, they produce identical patterns and thus, the same relia (Figure 10(e)). GASTON runs the fastest (Figure 10(f)) except on dataset Trans, which contains only a small number of patterns (see Exp-4, pp. 11), resulting in fast mining time for all methods. Since GASTON achieves good efficiency without compromising accuracy, we used it as our default.

(2) *Deep Q-learning.* Motif composition leads to a large state space. In the worst case, an excessive number of patterns can be composed from even a small set S of motifs, e.g., if we use a naive $\otimes_{\text{all}}(\cdot)$ and there are M pairs of mergeable vertices in S , $\otimes_{\text{all}}(S)$ can have $O(2^M)$ patterns (pp. 5). To address this, we adopt deep Q-learning (DQN), which handles large state spaces efficiently and models complex structures with neural networks for generalization. In other words, we adopt DQN to strike a balance between exactness and scalability (pp. 6).

We have performed an ablation study testing a NEX variant without DQN in both original and revised submissions. DQN accelerates the discovery by $1.93\times$ (Figure 10(b), pp. 11).

(3) *Predicate generation.* Predicates are fundamental abstraction in dependency theory and underlie major dependencies, e.g., entity enhancing rules (REEs) [7], denial constraints (DCs) [13], [14], and graph functional dependencies (GFDs) [15]. Following established practice, the predicate space $\mathcal{P}$ is pre-built prior to enumerating its elements to form dependencies, e.g., [13] builds $\mathcal{P}$ by pairing all compatible attributes.

However, instead of generating all possible predicates, it is suggested in [14] that "restricting the predicate space helps prune meaningless results and reduces computational costs". To this end, various techniques have been developed to restrict $\mathcal{P}$. For example, [7] trains an ML model to retrieve predicates that are *correlated* to the consequences and [15] selects a set Γ of *active* attributes and only considers predicates composed of attributes in Γ . Although these methods are also heuristic, they are proven effective in practice. Since our goal is to explain negative GNN predictions, we only keep predicates in $\mathcal{P}$ if they are satisfied frequently in the negative classes (pp. 7).

As suggested, we have added an ablation study comparing a variant of NEX that uses all possible predicates as $\mathcal{P}$. As shown in Figure 10(b) (pp. 11-12), this variant cannot finish within 24 hours, verifying the benefit of restricting the predicate space.

(4) *Pruning based on anti-monotonicity.* Anti-monotonicity is a property widely adopted in existing rule discovery algorithms, e.g., [13], [7], [15], [16], to prune the search space. We respectfully clarify that same as existing studies, our pruning based on the anti-monotonicity of support is *not* heuristic-based, i.e., it guarantees the rules we prune are invalid (i.e., not σ -frequent). While we did not claim that the anti-monotonicity-based pruning as our major contribution, it accelerates the discovery process without missing any outputs.

(5) *Support and confidence*. Threshold-based strategies (e.g., support and confidence thresholds) are commonly used in rule discovery to identify frequently applicable and reliable rules while filtering out irrelevant ones [7], [15], [16]. We have provided a guidance (pp. 7) for their setting. Users may rely on data statistics for initial settings, enabling fast discovery of a preliminary rule set. These parameters can then be iteratively fine-tuned using incremental rule discovery [17].

While a complete treatment of incremental tuning is beyond the scope of this paper, we outline the strategy for the incremental discovery of RxGNNs in response to parameter updates in [2] for the lack of space. Besides, we have also tested the impact of support/confidence in Section VI (pp. 12).

In summary, as suggested, we have added justification for major design choices (e.g., DQN and restricted predicate space, pp. 5-7) and substantially revised the ablation study to validate the usefulness of each component in NEX (pp. 12), including (a) clustering, (b) candidate space, (c) DQN, (d) restricted predicate space, (e) predicate orderings, (f) LLMs, (g) pattern mining, (h) general patterns and (i) 1-WL predicates.

[R402 & Q12] *A key issue is the usage of blackbox LLMs in their predicate ordering. This is a critical part of the framework. This seems like a critical detail swept under the rug. What LLM is used? How does choice of LLM change PPL ordering? Without extended details of the choice of LLM how to ensure reproducibility? How can this framework aid interpretability if a key component is outsourced to a blackbox?*

Specifically, as stated prior a black box LLM is incorporated to rank predicate ordering without further details on how different LLM choices can affect their outcome.

[A] Due to the fast development of ML (and LLM) techniques, ML-guided data management has shown great success in various domains, from indexing [18] to scheduling [19] (pp. 6). As highlighted in [18], “the idea of replacing core components of a data management system through learned models has far reaching implications for future systems designs”.

While we use an LLM to guide the predicate ordering, we respectfully clarify that it is a *speedup optimization* that does not affect the rule quality, i.e., it will find exactly the same set of preconditions, *no matter what ordering is used*. In other words, the predicate ordering is *not* an essential component for the interpretability of our framework, and our interpretability remains *intact*, with or without this optimization. We have clarified this (pp. 7). Nevertheless, a good ordering allows the anti-monotonicity pruning (pp. 5) to be applied earlier. Please also see our response to **[R105]** for concrete examples.

As suggested, we have also made the following changes:

- (1) We have added details about LLMs, e.g., Qwen2.5 [20] as default LLM (pp. 10) and an example prompt on Claim in [2].
- (2) We have added an experiment that evaluates the impact of LLMs on (a) rule discovery time and (b) actual orderings, using Qwen2.5, Gemma3 [21], Llama3.2 [22] and Phi3 [23] (pp. 11-12). Due to their strong capabilities, all LLMs generally

provide reasonable orders, leading to similar rule discovery times (Figure 10(d)), but their actual orderings vary slightly.

To compare, we computed a ground truth (GT) order on Claim based on predicate frequencies in final valid rules (for limited space, see the case study in [2] for concrete predicates). We compared the top-5 predicates from each LLM, visualized using the heatmap in Figures 10(c), where darker cells indicates higher GT ranks. As shown there, Qwen2.5 best matches the GT order, recovering all GT top-5 predicates and misordering only one predicate (p_4). Thus we adopted it as our default.

(3) In our original submission, we have conducted an ablation study by comparing a variant of NEX without the perplexity predicate ordering (PPO); this variant processes predicates based on the descending order of their supports following [7]. The use of PPO speeds up rule discovery by $1.21\times$.

In the revision, we have re-organized the ablation study and expanded the experiment for predicate orderings to a standalone test (Figure 10(b) and pp. 12), by replacing PPO with (a) the support-based ordering (SO) and (b) a random ordering (RO). Note that all these orderings only differ in the resulting rule discovery time and they produce exactly the same set of RxGNNs. Thus, we only compared the efficiency of these orderings. Please also see our response to **[R105]**.

To provide more intuitive ideas, we have added a case study on Claim, which directly contrasts the orderings of PPO and SO. SO prioritizes general predicates in processing, rather than those that are truly discriminative, resulting in delays in pruning. For the lack of space, the case study is presented in [2].

[R403] *The usage of support confidence can encourage overly specific patterns to be selected and overfitting to occur? How do the authors deal with this particular scenario?*

[A] In practice, overly specific rules are naturally discouraged by the use of support and confidence together. High confidence alone may indeed favor highly specific rules, but requiring high support simultaneously ensures that a rule must hold across a sufficiently large portion of the data to be retained. A rule that is too specific cannot “cover” enough instances to achieve high support and is therefore filtered out automatically, mitigating overfitting in practice. Please also see our response to **[R401]**.

Moreover, our framework does not fix the thresholds rigidly. We have provided guidance (pp. 7) on incrementally tuning support and confidence to balance coverage and reliability, allowing users to progressively avoid overspecialized patterns.

[R4 Availability] *Please provide more details on the partial predicate ordering and specifically what LLM is used and how reproducibility can be ensured with this blackbox component.*

[A] As suggested, we have provided codes for prompt generation in https://github.com/hdqlkncso24jvf/NEX/tree/main/NEX/llmsft/ppl_generator. For reproducibility, in the above folder, we provide the LLM settings used in our experiments as well as the actual predicate orderings used, so that readers can directly run rule discovery with the same orderings.

Many thanks for observing the novelty of the work!

Explaining GNN Negatives Globally and Locally

Kehan Pang¹, Wenfei Fan^{2,3,1}, Min Xie², Dandan Lin²

¹Beihang University ²Shenzhen Institute of Computing Sciences ³University of Edinburgh
pangkehan@buaa.edu.cn, wenfei@inf.ed.ac.uk, {xiemin, lindandan}@sics.ac.cn

Abstract—This paper studies explanations for graph neural network (GNN) classifiers $\mathcal{M}$ when $\mathcal{M}$ makes negative predictions, such as loan denials, paper rejections, or job application turn-downs. The objective is to (a) provide global explanations of $\mathcal{M}$'s behavior and (b) generate *counterfactual explanations* locally at a vertex u , suggesting the minimal changes to features or topology around u needed to flip $\mathcal{M}$'s prediction. We propose a class of rules that treat negative predictions as their consequences, and employ the 1-dimensional Weisfeiler–Leman (1-WL) test as a predicate, which is at least as expressive as GNN classifiers. We develop algorithms to (a) learn such rules for global explanations and (b) apply them to compute local counterfactuals. Extensive experiments on real-world graphs show that our approach outperforms state-of-the-art methods across major metrics.

I. INTRODUCTION

When an ML model $\mathcal{M}$ makes negative predictions, one naturally seeks explanations, since understanding the reasons behind refusal is often more critical than for positive outcomes. Why was my paper rejected, my loan denied, or my job application turned down? Moreover, what minimal changes would allow me to pass the model's check?

Explanations for $\mathcal{M}$'s predictions fall into two main types.

- *Global explanations* capture the *input-independent* behavior of $\mathcal{M}$, identifying structures and features behind negative predictions, to provide insights and build trust.
- *Local explanations* (a.k.a. *counterfactual explanations*) suggest minimal changes to a specific *input instance* to flip the prediction, such as paper acceptance or loan approval.

This paper focuses on both global and local explanations for Graph Neural Networks (GNNs), which are widely-used in recommendation [24], [25], fraud detection [26], [27] and drug discovery [28], [29], [30], [31], among others.

Prior work on explaining GNN negatives includes local [32], [33], [34], [35], [36], [37], [38], [39], [40], [41], [31] (surveyed in [42]), global [43], [44], [45], [46], [47], and hybrid approaches [48], [49]. However, these methods usually suggest subgraph modifications without clarifying which features drive predictions or under what conditions they hold. Moreover, they often propose making immutable changes, which is infeasible. Furthermore, none of them provides a unified logical framework for both global and local explanations.

Example 1: Consider a *real* job application case [50] (Figure 1), where an applicant u (ID 2445) is turned down by a GNN model $\mathcal{M}$. For clarity, we show only representative perturbations within the 2-hop neighborhood graph G_u of u .

Prior methods craft local counterfactuals by altering subgraph topology, often ignoring features and feasibility.

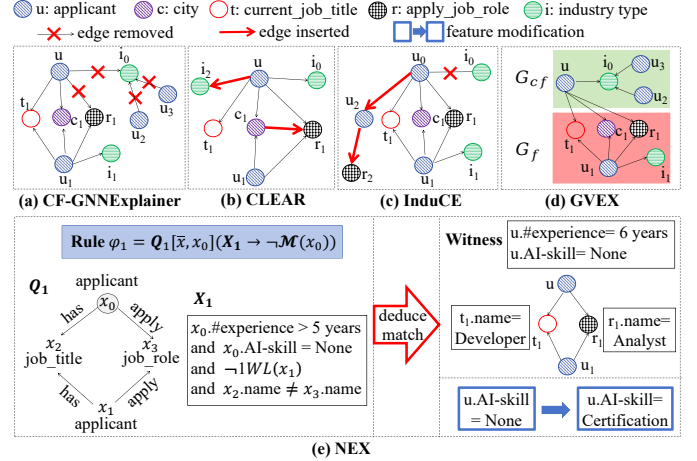


Fig. 1: A case study to swap prediction at u 's job application

- CF-GNNExplainer [34] (Figure 1(a)) removes multiple edges from G_u , e.g., $(u_2, \text{works_in}, i_0)$. However, since vertex features are ignored, the prediction is not flipped.
- CLEAR [37] (Figure 1(b)) adds new edges, such as (c_1, r_1) . While this changes the prediction, it violates domain constraints (e.g., a city c_1 cannot connect to a job-role r_1).
- InduCE [36] (Figure 1(c)) inserts an edge between two applicants and alters the job role of applicant u_2 , which is unrealistic; CFGraph [49] suffers from the same issue.
- GVEX [48] (Figure 1(d)) deletes a factual subgraph G_f from G_u to get G_{cf} , but G_{cf} cannot swap the prediction.

Overall, these methods only perturb edges but ignore feasible feature updates, leading to less effective explanations.

For global explanations, existing methods are limited primarily because they overlook vertex features. Specifically, (a) XGNN [43] and CFGraph [49] generate patterns with fake edges; (b) GNNInterpreter [44] and GVEX [48] produce overly complex patterns that obscure the most discriminative substructures; and (c) GVEX also suffers from poor efficiency. $\square$

To address these, we present NEX, a unified logical framework for explaining GNN negatives *globally* and *locally*. NEX introduces RxGNNs (Rules for eXplaining GNN classifiers), a new class of logic rules for explanations (see Section II).

Given a GNN $\mathcal{M}$ and a vertex variable x_0 with $\mathcal{M}(x_0) = \text{false}$, denoted as $\neg \mathcal{M}(x_0)$, NEX mines RxGNNs of the form $Q(X \rightarrow \neg \mathcal{M}(x_0))$, where Q is a graph pattern pivoted at x_0 , and X is a set of predicates on vertex features. Intuitively, Q captures topological structures around x_0 , and X specifies features conditions. The rules both yield global explanations by revealing structures/features most responsible for $\neg \mathcal{M}(x_0)$,

and derive local explanations for vertices u in graph G by identifying specific matches of Q and X around u as *witnesses*; counterfactuals are deduced by perturbing these witnesses.

Example 2: Continuing Example 1, Figure 1(e) shows a mined RxGNN φ_1 and its witness at applicant u in G [50]. Rule φ_1 consists of a graph pattern Q_1 and a precondition X_1 on vertex features. The witness explains why $\neg\mathcal{M}_1(u)$: (a) u 's job title is developer while applying for analyst, indicating a career change; (b) another applicant u_1 shares the same title and applies the same role as u , and u_1 has already been denied (by the 1-WL test [51], [52], see Section II-A); and (c) although u has sufficient experience (> 5 years), u lacks AI-skill.

By perturbing this witness, NEX suggests u get an AI-skill certification. This counterfactual explanation is feasible for u to implement and more importantly, it flips $\mathcal{M}_1$'s prediction.

Unlike prior local methods (Example 1), φ_1 pinpoints which structures and features drive the negative prediction and under what conditions. It then derives a feasible counterfactual (e.g., by updating mutable features) rather than simply altering graph topology. It also makes less perturbations than prior methods.

Unlike prior global methods, RxGNN φ_1 not only accounts for individual features, but also highlights the most discriminative substructure, containing only real edges in G . $\square$

Contributions & organization. We propose a uniform framework for both global and local explanations of GNN classifiers.

(1) *Rules for explaining GNNs (RxGNNs)* (Section II). We propose RxGNNs for explaining GNNs. Unlike prior rules, RxGNNs (a) support rich predicates, including negation and the 1-WL test [51], [52] (as expressive as GNNs [53], [54]), to capture diverse behaviors of GNNs; (b) use general graph patterns to encode topology; and (c) treat negative predictions as consequences, enabling actionable input perturbations.

(2) *NEX: Negative prediction EXplanations* (Section III). We develop NEX, a unified framework for explaining GNN negatives. Given a classifier $\mathcal{M}$, NEX discovers a set Σ of RxGNNs as global explanations. For a given vertex u in graph G where $\mathcal{M}$ predicts negative, NEX derives a counterfactual explanation, the necessary changes for flipping $\mathcal{M}$'s decision at u .

(3) *Model-guided discovery* (Section IV). We develop an efficient, parallelly scalable [55] algorithm for discovering RxGNNs, guided by a given GNN $\mathcal{M}$. It first identifies motifs most responsible for $\mathcal{M}$'s predictions, and then composes them into graph patterns via a deep Q-learning policy [56] to avoid exponential search. Preconditions are subsequently generated with a novel predicate ordering to reduce computational cost.

(4) *Counterfactual explanations* (Section V). For a vertex u in graph G with $\neg\mathcal{M}(u)$, we seek a set of perturbations (including edge removals and feature modifications) that invalidates all RxGNNs in Σ at u , i.e., the minimal changes to flip $\mathcal{M}(u)$ from negative to positive. Although the problem is NP-hard, we develop a strategy to select cost-effective perturbations.

(5) *Experimental study* (Section VI). Using real-world graphs, we find (a) RxGNNs learned by NEX provide accurate global

explanations, with 70.16% recognizability (measuring how consistently the rules reflect the model behavior) and 189.68% higher reliability (measuring how well the rules cover testing vertices) than baselines on average. (b) NEX yields effective counterfactuals, with fidelity (success in flipping predictions) and sparsity (minimality of perturbations) improved by 33.60% and $3.25\times$, respectively, over prior methods. (c) NEX is efficient: achieving $3.06\times$ parallel speedup (16 vs. 4 processors) in rule discovery, and generating a counterfactual in 1.38s on a graph with 15.4M vertices and 12.7M edges.

Related work. We categorize the explanation methods (mostly on counterfactuals for GNN-based models) as follows.

(1) *Local counterfactual explanation.* Existing methods [32], [33], [34], [35], [36], [37], [38], [39], [40], [41], [31] (surveyed in [42]) generate counterfactuals by identifying minimal perturbations to the input graph that flip GNN predictions, e.g., CF-GNNExplainer [34] uses optimization on edge removals; RCExplainer [35] removes resilient edge subsets; CLEAR [37] employs a variational autoencoder; Induce [36] leverages reinforcement learning; and LocalCE [39] finds pairs of similar but differently classified vertices as counterfactuals.

(2) *Global explanation.* Global methods generate counterfactual graphs, including XGNN [43], which uses reinforcement learning; GNNInterpreter [44], which utilizes numerical optimization; DAG [45], which applies randomized greedy search; and GLGExplainer [57], which employs prototype learning. GCFExplainer [46] performs global counterfactual reasoning via random walks, and ComRecGC [47] generates counterfactual graphs relevant to all inputs. Although some methods (e.g., XGNN) do not take specific graphs as input, they still rely on pre-trained GNNs and thus implicitly depend on the training data. In contrast, NEX generates both global and local explanations and thus, naturally operates on a specific graph.

(3) *Hybrid.* GVEX [48] adopts a two-tier structure, combining global graph patterns with local subgraph explanations. CFGraph [49] derives counterfactuals by adding or removing edges and aggregates them into global explanations. The most related work, Makex [58], employs rule-based explanations for GNN link predictions but with less expressive rules, limited star patterns, and restricted predicates. It supports only factual explanations by identifying sufficient structures, whereas counterfactuals require graph modifications (see [2] for details).

(4) *Non-GNN explanations.* Logic-based explanation methods [59], [60], [61], [62], [63], [64] have been studied across various data modalities (see surveys [65], [66]). Systems such as Geco [67], FACET [68], Mace [69] and Dice [70] identify plausible feature changes to flip ML predictions. Alternative approaches employ declarative programming (e.g., answer-set programming [64]) or automated reasoning with SAT/SMT solvers [63]. In contrast, GNN explanations must account for graph topology, which these methods cannot capture.

Discussion. This work differs from prior work in several key aspects: (a) RxGNNs combine graph patterns with preconditions, revealing not only which structures are decisive but also

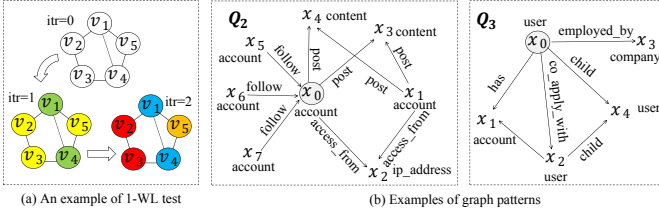


Fig. 2: Running examples

under what feature conditions predictions are made, beyond simple meta-paths or subgraphs used earlier; (b) we emphasize counterfactual explanations that flip predictions, unlike factual explanations that require no graph changes, general graph patterns, or negated predicates such as [58]; (c) we provide a unified framework for both global and local explanations applicable to any GNN classifier; and (d) unlike prior logic-based methods [59], [60], [61], [64], [63], which rely on constant predicates or manual preference settings, RxGNNs support diverse predicates (including negation and 1-WL test), enabling expressive reasoning for GNNs with automatic setup.

II. RULES FOR EXPLAINING GNN NEGATIVES

This section first reviews 1-WL test (Section II-A), and then defines rules for explaining negative predictions (Section II-B).

A. Graph Patterns and 1-WL Test

Assume two countably infinite sets of symbols Λ and Υ , for labels and vertex attributes (*i.e.*, vertex features), respectively.

Graphs. We consider directed labeled graphs $G = (V, E, L_G, F_A)$, where (a) V is a finite set of vertices; (b) $E \subseteq V \times \Lambda \times V$ is a finite set of labeled edges, with $e = (v, l, v')$ denoting an edge from v to v' labeled $l \in \Lambda$; (c) each vertex $v \in V$ has a label $L_G(v) \in \Lambda$ and a tuple of attributes $F_A(v) = (A_1 = a_1, \dots, A_n = a_n)$, where $A_i \in \Upsilon$, a_i is a constant, and $v.A_i = a_i$ (with $A_i \neq A_j$ if $i \neq j$). Different vertices may carry different attributes, while edges carry only labels.

Patterns. A (connected) *graph pattern* is $Q[\bar{x}, x_0] = (V_Q, E_Q, L_Q)$, where (1) V_Q and E_Q are finite sets of pattern vertices and edges, respectively; (2) L_Q assigns a label $L_Q(v) \in \Lambda$ to each vertex $v \in V_Q$; (3) $\bar{x}$ is a list of distinct variables, each $x \in \bar{x}$ denoting a vertex $v \in V_Q$ [71], [72]; and (4) x_0 is a designated variable in $\bar{x}$, representing the entity of interest.

We denote variables in Q by x, y , and vertices in G by u, v .

Example 3: The graph pattern $Q_1[\bar{x}, x_0]$ in Figure 1(e) models entities and relations in the graph of real-life job applications [50]. It has four vertex variables $\{x_0, x_1, x_2, x_3\}$ labeled as applicant, job_title and job_role, and four edges; *e.g.*, (x_0, apply, x_3) denotes that applicant x_0 applies for job x_3 . $\square$

Pattern matching. Following [73], [74], a *match* of a pattern $Q[\bar{x}, x_0]$ in a graph G is a homomorphism $h : Q \rightarrow G$ such that: (a) for each vertex $v \in V_Q$, $L_Q(v) = L_G(h(v))$; and (b) for each edge $e = (v, l, v') \in E_Q$, $(h(v), l, h(v')) \in E$.

GNNs. Graph neural networks (GNNs) [75], [76], [77], [78] leverage graph topology and vertex features to learn embeddings for vertex classification. Let $\mathbf{e}_v$ denote the initial feature vector of vertex v obtained from its attributes. An L -layer

Notations	Definitions
$G, Q[\bar{x}, x_0]$	graph, graph pattern
$\neg \mathcal{M}(x_0)$	a GNN model that predicts false at a vertex x_0
$\neg 1\text{WL}(x)$	the (negated) predicate for 1-WL test
p, X	a predicate, precondition (collection of predicates)
φ	an RxGNN $\varphi = Q[\bar{x}, x_0](X \rightarrow \neg \mathcal{M}(x_0))$
Σ	a set of RxGNNs
σ, δ, N	thresholds of support and confidence, max pattern size
$\text{supp}(), \text{conf}()$	the support and confidence of an RxGNN φ on graph G
$\mathbb{O}, G \oplus \mathbb{O}$	a set of perturbations in G , the graph G perturbed by $\mathbb{O}$
$\text{cost}(v), \text{eff}(v)$	the cost and effectiveness of a vertex v

TABLE I: Notations

GNN iteratively refines $\mathbf{e}_v$ by aggregating embeddings of its L -hop neighbors. At layer t , the embedding of v is:

$$\mathbf{e}_v^t = \text{Update}(\mathbf{e}_v^{t-1}, \text{Aggregate}(\{\mathbf{e}_{v'}^{t-1} \mid v' \in \mathcal{N}(v)\})).$$

Here Aggregate and Update denote the aggregation and update functions, respectively, and $\mathcal{N}(v)$ is the 1-hop neighborhood of v . After L iterations, the model outputs the final embedding $\mathbf{e}_v^L$; its classification probability is given by a function $f(\mathbf{e}_v^L)$ learned *w.r.t.* the training classes. For simplicity, we follow [78], [60] and study binary vertex classification: given a threshold λ , $\mathcal{M}(v)$ returns true if $f(\mathbf{e}_v^L) \geq \lambda$ and false otherwise.

1-WL test. We review the 1-dimensional Weisfeiler-Leman (1-WL) test [51]. Given a graph G where each vertex is initialized with a color (from its feature vector), 1-WL iteratively refines colors [54]: for vertices v and v' of the same current color, they receive different new colors if they differ in the number of neighbors of some color d . The process iterates until stable, and vertices sharing the same final color form a class.

Example 4: Consider Figure 2(a), an example borrowed from [79]. All vertices start white, and colors are refined iteratively. In the first iteration, v_1 and v_5 receive different colors as they have 3 and 2 white children, respectively. After two iterations the process stabilizes, with v_2 and v_3 sharing the same final color and thus belonging to the same class. $\square$

Notations. We will use the following notations (see Table I).

(1) We use $\neg \mathcal{M}(x)$ to denote a negative prediction (false) of model $\mathcal{M}$ at a vertex v , *i.e.*, $f(\mathbf{e}_v^L) < \lambda$ for threshold λ .

(2) We treat $\neg 1\text{WL}(x)$ as a predicate, which holds if there is a vertex y such that $\neg \mathcal{M}(y)$ holds, and x and y belong to the same 1-WL class. **Intuitively, if x and y lie in the same 1-WL class, they exhibit high topological similarity. Since most existing GNNs are no more expressive than the 1-WL test [3], [4], [5], [6], if $\mathcal{M}$ predicts false at y , it is likely to do so for x as well. Thus, the 1-WL test as a predicate equips our rules with sufficient expressive capability to characterize and explain the behavior patterns of GNN models. The 1-WL can be computed in $O((|V| + |E|) \log |V|)$ time [54].**

B. Rules with Negated Predicates

We next define RxGNNs with $\neg \mathcal{M}(x)$ and $\neg 1\text{WL}(x)$.

Predicates. A *logic predicate* of a pattern $Q[\bar{x}, x_0]$ is:

$$p ::= x.A \oplus y.B \mid x.A \oplus c \mid \neg 1\text{WL}(x).$$

Here $\oplus \in \{=, \neq, <, \leq, >, \geq\}$; variables $x, y \in \bar{x}$; c is a constant; A, B are attributes; and $\neg 1\text{WL}(x)$ is the 1-WL predicate.

RxGNNs. Given a GNN $\mathcal{M}$, a *Rule* φ for *eXplaining* $\mathcal{M}$ is

$$Q[\bar{x}, x_0](X \rightarrow \neg \mathcal{M}(x_0)),$$

where $Q[\bar{x}, x_0]$ is a graph pattern and X is a conjunction of predicates. We refer to Q and $X \rightarrow \neg \mathcal{M}(x_0)$ as the *pattern* and *dependency* of φ , respectively, and X is its *precondition* and $\neg \mathcal{M}(x_0)$ is its *consequence*. We focus on RxGNNs with the same consequence $\mathcal{M}$, i.e., RxGNNs pertaining to $\mathcal{M}$.

One can plug any pretrained GNN $\mathcal{M}$ into RxGNNs, such as GCN [80] or GAT [75], as long as $\mathcal{M}$ is *fixed* (its architecture and weights no longer change), and *deterministic* (it produces the same output for the same input) [81], [48], [31], [39].

Intuitively, the pattern $Q[\bar{x}, x_0]$ captures the topology around vertex variable x_0 that $\mathcal{M}$ inspects, while the precondition X encodes interactions among vertex features. Together, Q and X explain why $\mathcal{M}(x_0)$ makes its prediction. We separate Q and X to (a) visualize entity topology via Q , and (b) accelerate evaluation by exploiting the locality of pattern matching.

Example 5: Below are RxGNNs with patterns in Figure 2(b).

(1) An RxGNN φ_1 is given in Example 1 to explain why $\mathcal{M}_1$ predicts that the job application of x_0 is declined.

(2) RxGNN $\varphi_2 = Q_2[\bar{x}, x_0](X_2 \rightarrow \neg \mathcal{M}_2(x_0))$, where X_2 is $\neg 1\text{WL}(x_1) \wedge x_3.\text{content} = x_4.\text{content} \wedge x_5.\text{account_age} < 24\text{h} \wedge x_6.\text{account_age} < 24\text{h} \wedge x_7.\text{account_age} < 24\text{h}$. It says that $\mathcal{M}_2$ flags an account x_0 for suspension since (a) x_0 behaves like **a suspended account x_1 (judged by 1-WL test)**, e.g., both post repetitive contents such as spammy “Great post!”, and they have been accessed from the same IP; and (b) many followers of x_0 are created within 24 hours (bot-like accounts).

(3) RxGNN $\varphi_3 = Q_3[\bar{x}, x_0](X_3 \rightarrow \neg \mathcal{M}_3(x_0))$, where X_3 is $x_0.\text{debt_to_income} \geq 40\% \wedge x_3.\text{default_history} = \text{yes} \wedge \neg 1\text{WL}(x_4) \wedge x_2.\text{credit} \leq x_4.\text{credit} \wedge x_1.\text{balance} < 10\text{K}$. It says that $\mathcal{M}_3$ denies the loan of x_0 because (a) x_0 has high debt-to-income ratio ($\geq 40\%$) and x_0 is employed by a company defaulted on a loan, (b) **an immediate relative x_4 of x_0 has already been denied (in the negative class by 1-WL test)**, (c) **the co-applicant x_2 of x_0 has worse credit than x_4 (in negative class)** and their joint account has low balance ($< 10\text{K}$). $\square$

Discussion. Graph rules have been studied for diverse tasks: (a) association analysis, e.g., GPARs [82], GARs [72] and TACOs [83]; (b) data cleaning, e.g., GFDs [71] and GCRs [84]; (c) enhancing ML recommendations, e.g., TIEs [85]; and (d) factual explanation, e.g., REPs [58]. Unlike these prior rules, RxGNNs exhibit three essential properties for expressive counterfactual explanations (see expressivity comparison in [2]):

(1) *Rich predicates.* RxGNNs support not only constant, variable, and ML predicates, but also (a) limited negation, including $\neg \mathcal{M}(x_0)$, $\neg 1\text{WL}(x)$, and $\neg(x.A \oplus y.B)$ via the dual of $\oplus$ (e.g., $\geq$ vs. $<$), which is crucial for modeling conditions that flip predictions; and (b) the 1-WL test as a predicate, which explicitly simulates message passing in GNNs through color refinement. Since 1-WL is at least as expressive as GNN classifiers, RxGNNs can faithfully explain $\mathcal{M}$ ’s predictions.

(2) *General patterns.* RxGNNs are defined on general graph patterns, which capture richer and more complex topological

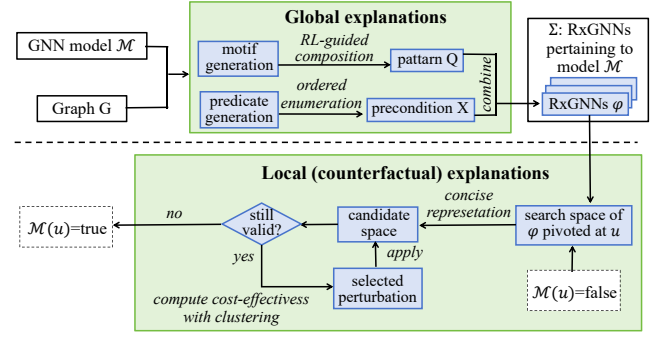


Fig. 3: The framework of NEX

structures than the star-shaped patterns used in [84], [85], [58].

(3) *Negated consequence.* RxGNNs take $\neg \mathcal{M}(x_0)$ as consequences, enabling counterfactual reasoning.

Semantics. Consider a graph G , an RxGNN $\varphi = Q[\bar{x}, x_0](X \rightarrow \neg \mathcal{M}(x_0))$, and a match h of the pattern Q in graph G .

We say that a match h satisfies a predicate p of $Q[\bar{x}, x_0]$, written $h \models p$, if: (a) $p = x.A \oplus y.B$: vertices $h(x)$ and $h(y)$ carry attributes A and B , and $h(x).A \oplus h(y).B$ (similarly for $x.A \oplus c$); (b) $p = \neg 1\text{WL}(x)$: the 1-WL test places $h(x)$ in the same class as some vertex v with $\mathcal{M}(v) = \text{false}$; and (c) $p = \neg \mathcal{M}(x_0)$: $\mathcal{M}$ predicts negative at $h(x_0)$.

We write $h \models X$ if h satisfies all predicates in X , and $h \models \varphi$ if $h \models X$ implies $h \models \neg \mathcal{M}(x_0)$. For a vertex $u \in G$, h is a *witness* of φ pivoted at u if $h \models \varphi$ and $h(x_0) = u$. RxGNN φ is *applicable* at u if there exists such a witness.

A graph G *satisfies* φ , written $G \models \varphi$, if every match h of $Q[\bar{x}, x_0]$ in G satisfies φ whenever $h \models X$. For a set Σ of RxGNNs, $G \models \Sigma$ means $G \models \varphi$ for all $\varphi \in \Sigma$.

III. GLOBAL AND LOCAL EXPLANATIONS

NEX provides *global* and *local* explanations with RxGNNs.

Global explanations. Given a GNN $\mathcal{M}$ and a graph G , NEX discovers a set Σ of RxGNNs for $\mathcal{M}$ ’s negative predictions. Each RxGNN is composed of important substructures, called *motifs*, since such motifs can drive prediction signals [40] (e.g., a Nitrobenzene structure often indicates mutagenicity). Guided by these signals, the rules in Σ capture instance-independent behaviors of $\mathcal{M}$ and highlight discriminative structures and features. Thus, Σ serves as the global explanation of $\mathcal{M}$.

Local explanations. For each vertex u with $\mathcal{M}(u) = \text{false}$, we derive a counterfactual explanation using Σ . Note that Σ is discovered offline and can be reused for different inputs u .

Counterfactual explanations. Given a witness h of φ mapping variable x to v in G , a *perturbation* is either (a) a *feature modification*, changing an attribute of v so that a predicate $p \in X$ involving x is no longer satisfied, or (b) an *edge removal*, deleting an incident edge of v so that h no longer matches Q . These two perturbations affect predictions in distinct ways.

A *counterfactual explanation* for $\neg \mathcal{M}(u)$ is the minimum set $\odot$ of perturbations such that in the updated graph, denoted as $G \ominus \odot$, no $\varphi \in \Sigma$ is applicable at u . Intuitively, $\odot$ specifies the *necessary changes* to flip $\mathcal{M}(u)$ from negative to positive.

Example 6: In Example 5, RxGNN φ_3 forms part of a global explanation for unsuccessful loan applications by capturing employment ties (e.g., $(x_0, \text{employed_by}, x_3)$), credit history (e.g., $x_2.\text{credit}$), and social connections (e.g., (x_0, child, x_4)).

For local explanation, a witness h_3 of φ_3 at u may suggest pay off credit card debt or increase savings to invalidate the high debt-to-income ratio and low balance checking. $\square$

Workflow. The workflow of NEX is shown in Figure 3.

(1) *Global explanations.* Since RxGNNs consist of patterns Q and preconditions X , the rule discovery involves (a) pattern generation, which extracts motifs from graph G and composes them into Q ; and (b) precondition generation, which enumerates predicates levelwise to form X . To accelerate, *RL-guided composition* and *predicate ordering* are designed to accelerate pattern and precondition generation, respectively (Section IV).

(2) *Local explanations.* Given vertex u with $\neg\mathcal{M}(u)$, we find perturbations to flip $\mathcal{M}(u)$ to positive as follows. For each φ in Σ , we concisely represent its search space using a *candidate space* C_φ . Then, we continually compute the *cost-effectiveness* of the perturbations with *clustering* to decide the right perturbation to apply, until C_φ becomes *invalid*; this means φ is no longer applicable at u . We present the details in Section V.

Extension. NEX readily extends to multi-class GNNs. Specifically, consequence $\neg\mathcal{M}(x_0)$ in RxGNNs can be replaced with a predicate $\mathcal{M}(x_0) = i$, where $i \in [1, K]$ denotes the class predicted for x_0 by $\mathcal{M}$ ($K = 2$ is binary classification). In this setting, counterfactuals aim to change the predicted class of x_0 from i to another. We test the extension in Section VI.

IV. MODEL-GUIDED RULE DISCOVERY

To mine RxGNNs as global explanations, we develop a model-guided rule discovery framework based on *motifs*. We first formulate the discovery problem (Section IV-A), then describe how to identify and compose motifs (Section IV-B), and finally present the discovery algorithm (Section IV-C).

A. The Discovery Problem

We start with criteria to evaluate the quality of RxGNNs.

Support. This measures how often an RxGNN φ can be applied in graph G . Formally, the *support* of φ in G is defined as:

$$\text{supp}(\varphi, G) = \|\mathcal{Q}(\bar{x}, x_0, G, X \wedge \neg\mathcal{M}(x_0))\|,$$

where $\mathcal{Q}(\bar{x}, x_0, G, X \wedge \neg\mathcal{M}(x_0))$ denotes the set of vertices $h(x_0)$ for all matches h of Q in G such that $h \models X \wedge \neg\mathcal{M}(x_0)$. Intuitively, the higher $\text{supp}(\varphi, G)$ is, the more frequent φ can be applied to G . For an integer σ , an RxGNN φ is σ -frequent if $\text{supp}(\varphi, G) \geq \sigma$. When $\varphi = Q[\bar{x}, x_0](\emptyset \rightarrow \neg\mathcal{M}(x_0))$ (i.e., X is empty), we denote $\text{supp}(\varphi, G)$ by $\text{supp}(Q, G)$ for simplicity.

Anti-monotonicity. Given $\varphi = Q[\bar{x}, x_0](X \rightarrow \neg\mathcal{M}(x_0))$ and $\varphi' = Q'[\bar{x}, x_0](X' \rightarrow \neg\mathcal{M}(x_0))$, we say that Q *subsumes* Q' , denoted as $Q \ll Q'$, if for any G , if Q' has a match in G , Q also has a match in G , e.g., Q removes edges/vertices from Q' .

Similarly, we say that RxGNN φ *subsumes* φ' , denoted as $\varphi \preceq \varphi'$, if $Q \ll Q'$ and $X \subseteq X'$, i.e., φ has a less restrictive

pattern and precondition compared with φ' . This results in the well-known *anti-monotonicity* for search space pruning, i.e., if $\varphi \preceq \varphi'$, then for any graph G , $\text{supp}(\varphi, G) \geq \text{supp}(\varphi', G)$.

Confidence. It measures how strong the connection between precondition X and negative prediction is, formally defined as:

$$\text{conf}(\varphi, G) = \frac{\text{supp}(\varphi, G)}{\|\mathcal{Q}(\bar{x}, x_0, G, X)\|}.$$

For a threshold δ , an RxGNN φ is δ -confident if $\text{conf}(\varphi, G) \geq \delta$. If X of φ is empty, we denote $\text{conf}(\varphi, G)$ by $\text{conf}(Q, G)$. Note that confidence does not have the anti-monotonicity.

RxGNNs of interest. Denote by Σ_A the set of all σ -frequent and δ -confident RxGNNs on G pertaining to the GNN $\mathcal{M}$.

A *cover* Σ of Σ_A is a minimal subset satisfying: (a) every rule φ in Σ_A is logically implied by Σ (written $\Sigma \models \varphi$, i.e., for all graphs G' , if $G' \models \Sigma$ then $G' \models \varphi$); and moreover, (b) no rule φ in Σ is redundant (i.e., $\Sigma \setminus \{\varphi\} \not\models \varphi$). Thus, Σ includes only nontrivial and non-redundant RxGNNs.

We prune redundant rules and return a non-redundant *cover* Σ of Σ_A using standard implication-based techniques (see [2]). Moreover, we restrict RxGNNs to those composed from a set $\mathcal{S}_M$ of *motifs* that play essential roles in predictions.

Motif. We define a motif as a graph pattern $\pi[\bar{x}, x_0]$ with a designated vertex x_0 that occurs more often in the negative classes than in the positive classes of G . More formally, given a threshold η , $\pi[\bar{x}, x_0]$ is a motif if $\frac{n_t}{n_f} < \eta$, where n_t (resp. n_f) is the number of vertices $h(x_0)$ for all matches h of $\pi[\bar{x}, x_0]$ in graph G with $\mathcal{M}(h(x_0)) = \text{true}$ (resp. false). For an integer k , we say that a motif is k -bounded if $|\bar{x}| \leq k$.

We *compose* multiple motifs into a graph pattern $Q[\bar{x}, x_0]$ using a specified *composition function* $\otimes(\cdot)$. Given a set S of motifs, $\otimes(S)$ denotes the set of patterns composed from S .

Model-guided discovery. The problem is as follows.

- *Input:* A graph G , a GNN $\mathcal{M}$, a set $\mathcal{S}_M$ of k -bounded motifs, a composition function $\otimes(\cdot)$, a maximum number N of pattern vertices, support and confidence thresholds σ and δ .
- *Output:* A cover Σ of Σ_A such that for each $\varphi \in \Sigma_A$, (a) its Q is composed from $\mathcal{S}_M$ via $\otimes$, i.e., $Q \in \cup_{S \subseteq \mathcal{S}_M} \otimes(S)$, (b) $|\bar{x}| \leq N$, (c) $\text{supp}(\varphi, G) \geq \sigma$, and (d) $\text{conf}(\varphi, G) \geq \delta$.

Here N controls the discovery cost. While any choice of $\mathcal{S}_M$ and $\otimes(\cdot)$ is possible, we present in Section IV-B our tailored strategies for identifying/composing motifs to explain $\neg\mathcal{M}$.

B. Motif Identification and Composition

Both motif identification and composition are challenging:

- (1) *Exponential motifs.* Identifying discriminative structures decisive for $\mathcal{M}$'s predictions can be exponential in the worst case. Thus we need to strike a balance between the cost of generation and the discriminability of the motifs identified.
- (2) *Excessive compositions.* Motif composition leads to a large search space. For instance, if we use a naive $\otimes_{\text{all}}(\cdot)$ and there are M pairs of mergeable vertices in S , $\otimes_{\text{all}}(S)$ can have $O(2^M)$ patterns, most of which are redundant or useless.

To this end, we develop a bounded motif identification strat-

Input: $G, \mathcal{M}, \sigma$ as above, a maximum size k and a threshold η .
Output: A set $\mathcal{S}_{\mathcal{M}}$ of motifs.

1. $\mathcal{S}_{\mathcal{M}} := \emptyset; \mathcal{G} := \emptyset;$
2. **for** each vertex $v \in G$ such that $\mathcal{M}(v) = \text{false}$ **do**
3. $G_{\text{sub}}(v) := \text{extractor}(G, v); \mathcal{G} := \mathcal{G} \cup \{G_{\text{sub}}(v)\};$
4. $\mathcal{F} := \text{generator}(\mathcal{G}, \sigma, k);$
5. **for** each candidate pattern $\pi[\bar{x}, x_0] \in \mathcal{F}$ **do**
6. $n_t := \|\mathcal{Q}(\bar{x}, x_0, G, \mathcal{M}(x_0))\|; n_f := \|\mathcal{Q}(\bar{x}, x_0, G, \neg\mathcal{M}(x_0))\|;$
7. **if** $\frac{n_t}{n_f} < \eta$ and $\nexists \pi' \in \mathcal{S}_{\mathcal{M}}$ such that $\pi' \ll \pi$ **then**
8. $\mathcal{S}_{\mathcal{M}} := \mathcal{S}_{\mathcal{M}} \cup \{\pi[\bar{x}, x_0]\};$
9. **return** $\mathcal{S}_{\mathcal{M}};$

Fig. 4: Algorithm getMotifs

egy with pre-pruning and an ML-guided composition function.

Motif identification. To identify useful motifs for RxGNNs, we adopt an “explain-and-summarize” paradigm: we first extract high-quality subgraphs that best explain the GNN model, and then summarize them into motifs, as follows.

(1) *Explain:* A concise explanation subgraph $G_{\text{sub}}(v)$ for each v with $\neg\mathcal{M}(v)$ is extracted from G , such that $\mathcal{M}$ could reproduce its prediction in $G_{\text{sub}}(v)$. All such subgraphs are stored in a set $\mathcal{G}$. The extraction is handled by a primitive extractor.

(2) *Summarize:* We abstract common structures as motifs from the subgraphs in $\mathcal{G}$, such that each motif is σ -frequent and k -bounded, where the use of k is to balance cost and discriminability. To fit the need, we adopt a primitive generator, which produces candidate patterns from $\mathcal{G}$ for later verification.

Algorithm. Given as input a graph G , a GNN model $\mathcal{M}$, a support threshold σ , the maximum size k and a threshold η , we outline the algorithm in Figure 4. Initializing $\mathcal{S}_{\mathcal{M}} = \emptyset$ (line 1), we extract a subgraph for each negative vertex $v \in G$ via the primitive extractor (lines 2-3). From these subgraphs, we mine σ -frequent and k -bounded patterns π via the primitive generator (line 4). Each candidate π is then evaluated by comparing its occurrence across positive and negative classes (lines 5-8). More specifically, we calculate the ratio $\frac{n_t}{n_f}$ of π , where n_t and n_f are defined in Section IV-A. We add π to $\mathcal{S}_{\mathcal{M}}$ if $\frac{n_t}{n_f} < \eta$ and π is not subsumed by any existing π' in $\mathcal{S}_{\mathcal{M}}$ (lines 7-8); intuitively, this is to pre-prune any redundant motif in $\mathcal{S}_{\mathcal{M}}$.

Implementation of primitives. To extract concise subgraphs that retain critical message flows for GNNs, we apply a graph mask over edges following [86]. For motif generation, various techniques can be used for exact pattern mining [87]. We adopt GASTON [9] to mine σ -frequent and k -bounded motifs from the masked subgraphs for its efficiency (see Section VI).

Composition function. We first define the composition of two motifs; similarly for multiple motifs. Given $\pi_1 = (V_1, E_1, L_1)$ and $\pi_2 = (V_2, E_2, L_2)$, a pattern $Q[\bar{x}, x_0] = (V_Q, E_Q, L_Q)$ is composed from π_1 and π_2 via $\otimes(\cdot)$ if V_Q (resp. E_Q) is obtained by merging vertices (resp. edges) in $V_1 \cup V_2$ (resp. $E_1 \cup E_2$), while L_Q inherits label assignments from π_1 and π_2 . Note that only vertices with the same label are mergeable.

Given a subset S of motifs and a support threshold σ , our composition function $\otimes(\cdot)$ returns $Q^* = \arg \max \{\text{conf}(Q, G) \mid Q \in \otimes_{\text{all}}(S) \text{ and } \text{supp}(Q, G) \geq \sigma\}$. To avoid excessive

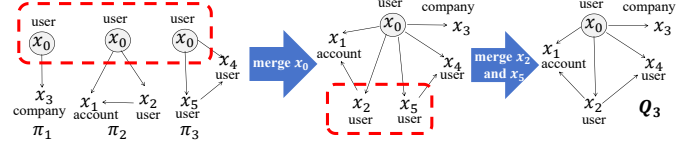


Fig. 5: An example of how Q_3 is composed

compositions, we learn a robust policy to guide this process, leveraging the recent success of ML-guided search [18], [19].

Overview. We guide motif composition with deep Q-learning (DQN) [56], which handles large space efficiently and models complex structures with neural networks for generalization, to strike a balance between exactness and scalability. It learns a policy to generate patterns by iteratively merging vertices.

Training. Our policy is learned with a specified reward function, which is defined as the confidence gain before and after composition: for a current pattern Q_i and a new pattern Q_{i+1} obtained by merging a pair of vertices in Q_i , the reward r is:

$$r = \begin{cases} -1 & \text{if } \text{supp}(Q_{i+1}, G) < \sigma, \\ \text{conf}(Q_{i+1}, G) - \text{conf}(Q_i, G) & \text{if } \text{supp}(Q_{i+1}, G) \geq \sigma. \end{cases}$$

We also adopt a special action [end] to terminate when $\text{supp}(Q, G) < \sigma$. Further details of DQN are provided in [2].

Inference. Once the policy is learned, it starts with an initial Q_0 (formed by merging all x_0 of motifs in S), and iteratively merges vertices with the highest reward until termination.

Example 7: Figure 5 illustrates how Q_3 in Figure 2(b) is generated (edge labels omitted). We identify three 3-bounded motifs, π_1 , π_2 , and π_3 , capturing x_0 ’s employment ties, financial status, and social connections, respectively. These motifs are then composed by merging all designated x_0 vertices into one, followed by merging user vertices x_2 and x_5 . $\square$

C. Parallel Discovery Algorithm

We present RxGNNMiner, our parallel discovery algorithm. Its novelties are: (a) leveraging motifs and composition functions for pattern generation, and (b) enforcing a predicate ordering for multi-thread parallelism in precondition generation.

Algorithm. As shown in Figure 6, RxGNNMiner first initializes the set $\mathcal{C}$ of graph patterns and the result set Σ (line 1). Then it composes the motifs in $\mathcal{S}_{\mathcal{M}}$ into patterns in $\mathcal{C}$ using $\otimes(\cdot)$, via procedure genPatterns (line 2, see below). For each resulting Q in $\mathcal{C}$, we compute X in a levelwise manner (lines 3-13). More specifically, we first generate the predicate space $\mathcal{P}$ guided by $\mathcal{M}$ via procedure genPredicates (line 4, see below). Then we start with $X = \emptyset$ (line 5) and iteratively expand X with predicates in $\mathcal{P}$ (lines 6-13), by maintaining a work queue Φ (line 5). Each pair $\langle X, \mathcal{P} \rangle$ in Φ is a computational task, which has three cases: (a) If $\varphi = Q(X \rightarrow \neg\mathcal{M}(x_0))$ is σ -frequent and δ -confident, it is valid and added to Σ ; here X is not expanded further since we want “minimal” RxGNNs only (lines 8-9). (b) If φ is σ -frequent but not δ -confident, we expand X with a new predicate from $\mathcal{P}$ (lines 10-13), yielding a new task in Φ . Unlike conventional expansion [15], we order predicates in $\mathcal{P}$ by perplexity-based scores (line 11, see below). (c) If φ is not σ -frequent, we directly skip it, since all RxGNNs

Algorithm RxGNNMiner

Input: $G, \mathcal{M}, S_{\mathcal{M}}, \otimes(\cdot), N, \sigma, \delta$ as stated above.

Output: A set Σ of RxGNNs pertaining to model $\mathcal{M}$.

```

1.  $\mathcal{C} := \emptyset; \Sigma := \emptyset;$ 
2.  $\mathcal{C} := \text{genPatterns}(\emptyset, S_{\mathcal{M}}, \mathcal{C}, \otimes, \sigma);$  /* Pattern generation */
3. for each pattern  $Q$  in  $\mathcal{C}$  do /* Precondition generation */
4.    $\mathcal{P} := \text{genPredicates}(Q, \mathcal{M});$ 
5.    $\Phi :=$  an empty work queue;  $X := \emptyset, \Phi.add(\langle X, \mathcal{P} \rangle);$ 
6.   while  $\Phi \neq \emptyset$  do
7.      $\langle X, \mathcal{P} \rangle := \Phi.pop(); \varphi := Q[\bar{x}, x_0](X \rightarrow \neg \mathcal{M}(x_0));$ 
8.     if  $\varphi$  is  $\sigma$ -frequent and  $\delta$ -confident then /* Checked at each thread */
9.        $\Sigma := \Sigma \cup \{\varphi\};$ 
10.    elseif  $\varphi$  is  $\sigma$ -frequent but not  $\delta$ -confident then
11.      for each  $p$  in  $\mathcal{P}$ , ordered by their ranking scores do
12.         $\mathcal{P}_{\text{new}} :=$  the set of all predicates in  $\mathcal{P}$  ordered after  $p$ ;
13.         $\Phi.add(\langle X \cup \{p\}, \mathcal{P}_{\text{new}} \rangle);$ 
14. return  $\text{cover}(\Sigma);$ 
```

Procedure genPatterns

Input: S_{sel} (selected motifs), S_{re} (remaining motifs), $\mathcal{C}, \otimes(\cdot)$, and σ .

Output: An updated set $\mathcal{C}$ of graph patterns.

```

15. if  $Q = \otimes(S_{\text{sel}})$  is  $\sigma$ -frequent then
16.    $\mathcal{C} := \mathcal{C} \cup \{Q\};$ 
17.   for each  $\pi$  in  $S_{\text{re}}$  do
18.      $\mathcal{C} := \mathcal{C} \cup \text{genPatterns}(S_{\text{sel}} \cup \{\pi\}, S_{\text{re}} \setminus \{\pi\}, \mathcal{C}, \otimes, \sigma);$ 
19. return  $\mathcal{C};$ 
```

Fig. 6: Algorithm RxGNNMiner

expanded from φ are not σ -frequent by the anti-monotonicity. Finally, when all tasks in Φ are done, a cover set Σ of non-redundant rules is returned [15] (line 14, see details in [2]).

Example 8: Assume that for pattern Q_3 , the set $\mathcal{P}$ has all predicates in X_3 (Example 5), ordered by the occurrence. Initially, $X = \emptyset$ and we check $\varphi' = Q_3(\emptyset \rightarrow \neg \mathcal{M}_3(x_0))$. If φ' is σ -frequent but not δ -confident, we expand precondition X with the next predicate $x_0.\text{debt_to_income} \geq 40\%$ and check the rule again. This process continues until φ_3 is found. $\square$

Procedure genPatterns. We generate the set $\mathcal{C}$ of patterns recursively by depth-first search with anti-monotonicity pruning (lines 15-19). We maintain two sets: (a) a set S_{sel} of selected motifs and (b) a set S_{re} of remaining motifs. Initially, $S_{\text{sel}} = \emptyset$ and $S_{\text{re}} = S_{\mathcal{M}}$ (line 2). In each recursion, we check whether $Q = \otimes(S_{\text{sel}})$ is σ -frequent (line 15). If so, we add Q into $\mathcal{C}$ (line 16) and recursively call **genPatterns** by expanding S_{sel} with a new motif from S_{re} (lines 17-18). If Q is not σ -frequent, the recursion stops and current $\mathcal{C}$ is returned since exploring more motifs can only lower the support by anti-monotonicity.

Procedure genPredicates. To avoid exhaustively generating all possible predicates in $\mathcal{P}$, we follow the established practice to restrict $\mathcal{P}$ since it “helps prune meaningless results and reduces computational costs” [14]. More specifically, we only keep predicates p closely related to $\neg \mathcal{M}$, i.e., p is kept in $\mathcal{P}$ if it is satisfied more frequently in the matches h of Q from negative classes than positive classes, w.r.t. a threshold (e.g., η).

Predicate ordering. We distribute workloads across processors for parallel processing [16], where multiple threads independently handle tasks. Since expansion halts once a non- σ -frequent φ is reached, predicate order affects efficiency.

To illustrate, consider two tasks $t_1 = \langle X_1, \mathcal{P}_1 \rangle$ and $t_2 = \langle X_2, \mathcal{P}_2 \rangle$, where $X_1 = \{p_1\}$ and $X_2 = \{p_2\}$. Denote the RxGNNs for t_1 and t_2 by ϕ_1 and ϕ_2 , respectively, and assume that ϕ_1 is σ -frequent but ϕ_2 is not. If t_1 is processed before t_2 , we may expand $X_1 = \{p_1\}$ to new $X_{1,2} = \{p_1, p_2\}$, yielding a new task $t_{1,2} = \langle X_{1,2}, \mathcal{P}_{1,2} \rangle$. However, we will not generate this task if we process t_2 first, since ϕ_2 is not σ -frequent.

This motivates us to enforce a predicate ordering that expands discriminative predicates first. **Note that the predicate ordering is a speedup optimization, i.e., it will find exactly the same set of preconditions, no matter what ordering is used.**

To assess how well a predicate discriminates $\neg \mathcal{M}(x_0)$, we leverage large language models (LLMs) for their rich prior knowledge. For each predicate p , we define its perplexity as:

$$\text{PPL}(p) \propto \exp(-\log P_{\text{LLM}}(\text{nlp}(p) \mid \text{nlp}(Q))),$$

where $\text{nlp}(\cdot)$ is natural language description of Q or p , and P_{LLM} is a probability given by LLMs, indicating the likelihood that appending $\text{nlp}(p)$ into $\text{nlp}(Q)$ yields a statement consistent with $\neg \mathcal{M}(x_0)$. Intuitively, $\text{PPL}(p)$ measures the correlation between a predicate p and $\neg \mathcal{M}(x_0)$. A lower $\text{PPL}(p)$ means that the LLM is more confident in this correlation, and thus p should be prioritized during processing.

Example 9: Consider two predicates $p_1: x_0.\text{debt_to_income} \geq 40\%$ and $p_2: x_0.\text{gender} = \text{Male}$. The simplified natural language description of Q_3 appended with p_1 (resp. p_2) is $T_1 = \text{“Given the topology in } Q_3, x_0\text{’s loan is denied since the debt to income ratio exceeds } 40\% \text{”}$ (resp. $T_2 = \text{“}\dots, x_0\text{’s loan is denied since he is a male”}$). Since T_1 is more reasonable, the LLM will assign lower perplexity to p_1 . $\square$

We can additionally factor in validation cost, favoring predicates strongly correlated with $\mathcal{M}$ while penalizing costly ones.

Remark. To choose parameters (thresholds σ, δ and motif size k), users may rely on data statistics for initial settings, enabling fast discovery of a preliminary rule set. These parameters can then be iteratively fine-tuned using incremental rule discovery [17], which naturally extends to RxGNNs (see [2]).

Recall that rule discovery is an offline process. To maintain the effectiveness of rules upon data updates, one can use an indicator that periodically samples new data to validate the current rule quality. If it detects a decline in the rule quality (e.g., if their support or confidence falls below a predetermined threshold), we re-run the offline rule discovery procedure, or use incremental discovery under small yet frequent updates.

Complexity. Given n processors, RxGNNMiner takes $O(\frac{1}{n} \times |\mathcal{S}_{\mathcal{M}}|^K \sum_{X \in \text{pow}(\mathcal{P})} |G|^{|X|})$ time, where K is the maximum number of motifs in $\mathcal{S}_{\mathcal{M}}$ that can compose a σ -frequent pattern ($K \ll |\mathcal{S}_{\mathcal{M}}|$), and $\text{pow}(\mathcal{P})$ is the power set of the predicate space $\mathcal{P}$. This is because (a) the computation is dominated by the validation of support and confidence, where the tasks are evenly distributed across n processors, and (b) it examines $\text{pow}(\mathcal{P})$ to generate X ’s for each Q at worst. This is the inherent cost for rule discovery [88], [13], [15], [16]. This said, we show that RxGNNMiner is parallelly scalable [55], reducing runtime nearly n -fold with n processors (see [2] for details).

V. LOCAL COUNTERFACTUAL EXPLANATIONS

This section presents our algorithm for counterfactual explanations. We begin by formulating the problem as follows.

- *Input*: A graph G , a GNN-based classifier $\mathcal{M}$, the set Σ of RxGNNs, and a negative vertex u in G with $\mathcal{M}(u) = \text{false}$.
- *Output*: The minimum set $\mathbb{O}$ of graph perturbations such that no $\varphi \in \Sigma$ is applicable at u in $G \ominus \mathbb{O}$. That is, after updating G with $\mathbb{O}$, $\mathcal{M}(u)$ is swapped from false to true.

Intuition. A brute-force approach would (i) compute all witnesses h of each RxGNN $\varphi \in \Sigma$ pivoted at u , and (ii) identify a perturbation set $\mathbb{O}$ such that every h ceases to satisfy X or Q of φ in $G \ominus \mathbb{O}$. However, this is costly, since even checking for the existence of matches of a general graph pattern Q is NP-hard [89]. Instead, we design an *inline* algorithm that identifies perturbations during the matching process itself.

The core of computing $Q[\bar{x}, x_0]$ matches is *vertex filtering*, which for each $x \in \bar{x}$, selects the set of candidate matching vertices in G , denoted as $C_Q(x)$, and discards those that cannot appear in any match. We extend vertex filtering to RxGNNs by requiring that candidates also satisfy X (Section V-A), and develop an algorithm to compute perturbations (Section V-B).

A. Candidate sets for RxGNNs

Along the same line as $C_Q(x)$ for pattern Q , given an RxGNN $\varphi = Q[\bar{x}, x_0](X \rightarrow \neg \mathcal{M}(x_0))$ and a vertex u in G , we define $C_\varphi^u(x)$ to be the *candidate set* of vertices in G that can be mapped to x in a witness h of φ pivoted at u (i.e., h is a match of pattern Q , $h \models \varphi$ and $h(x_0) = u$). When u is clear in the context, we simply write $C_\varphi^u(x)$ as $C_\varphi(x)$. Denote the union set of $C_\varphi(x)$ for all $x \in \bar{x}$ as the *candidate space* C_φ .

Same as $C_Q(x)$, we specify the required properties for C_φ :

- *General-pattern support*. It supports general patterns Q , not just star patterns or patterns with bounded tree-width [90].
- *PTIME cost*. Set C_φ is computable in PTIME, even though exact matching of general pattern Q is already NP-hard.
- *Soundness*. A candidate space C_φ is *sound* if for each witness h of φ where $h(x) = v$, v must be in $C_\varphi(x)$. As a compromise, C_φ may have *false positives*, i.e., for $v \in C_\varphi(x)$, there may exist no witness h of φ such that $h(x) = v$.

A naive $C_\varphi(x)$ is V , the set of all vertices in G . However, a good C_φ should be tight, with minimal false positives, to concisely represent the search space of φ . Below we first show how to make use of sound C_φ , followed by its construction.

Sufficient condition. Inspired by the *emptyset failure* [91], we devise a sufficient condition on sound C_φ to check whether a witness of φ exists at $u \in G$ (i.e., φ is applicable at u).

Lemma 1: *Given an RxGNN φ and a sound candidate space C_φ of φ , if there exists a pattern vertex x of φ such that $C_\varphi(x) = \emptyset$, then there exists no witness h of φ pivoted at u . □*

Proof. Assume that there is a witness h of φ pivoted at u in G ; then, by the soundness of C_φ , $h(x) \in C_\varphi(x)$ for each $x \in \bar{x}$, contradicting to $C_\varphi(x) = \emptyset$ for some x . □

We say a sound C_φ is *invalid* if it satisfies the conditions of

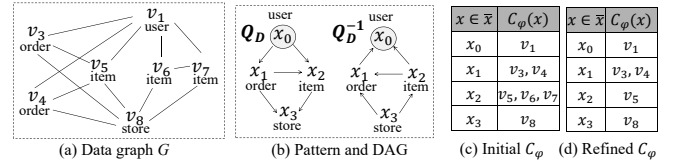


Fig. 7: Candidate space-based counterfactual explanation

Lemma 1. Intuitively, for pivot $u \in G$, if its C_φ is invalid, φ is not applicable at u . Thus, for each $\varphi \in \Sigma$, we compute C_φ and iteratively perturb G (and hence C_φ) until it becomes invalid.

Example 10: Consider a graph G and a φ with pattern Q_D in Figures 7(a)-(b) (edge labels omitted). Assume that φ has two witnesses at v_1 , i.e., $h_1: (x_0, x_1, x_2, x_3) \mapsto (v_1, v_4, v_5, v_8)$ and $h_2: (x_0, x_1, x_2, x_3) \mapsto (v_1, v_3, v_5, v_8)$. A sound and valid C_φ is given in Figure 7(d), since $C_\varphi(x) \neq \emptyset$ for all x in $\bar{x}$. □

New Problem. Based on these, it suffices to study a candidate space-based counterfactual explanation problem as follows.

- *Input*: G , $\mathcal{M}$, Σ , and u with $\mathcal{M}(u) = \text{false}$ as above.
- *Output*: A minimum set $\mathbb{O}$ of perturbations on G such that for each $\varphi \in \Sigma$, C_φ is *invalid* after updating G with $\mathbb{O}$.

Below we present a PTIME method for computing C_φ .

Candidate space construction. We construct C_φ by exploiting *DAG-graph dynamic programming* [91], to check topology (i.e., match Q) and features (i.e., satisfy X) simultaneously.

Overview. Given a graph G , a vertex u , an RxGNN $\varphi \in \Sigma$, we construct a candidate space C_φ pivoted at u in three steps.

(1) *Rooted DAG.* We first build a *rooted directed acyclic graph* (DAG) Q_D from the pattern $Q[\bar{x}, x_0]$ of φ , by traversing Q in BFS order from x_0 and directing all pattern edges from upper levels to lower levels. The DAG (a) contains no cycle, and (b) only root x_0 in G_D has no incoming edges (see [2]).

(2) *Initialization.* We initialize the candidate space C_φ pivoted at u by checking labels, degrees and predicates, i.e., $v \in C_\varphi(x)$ if $L_G(v) = L_Q(x)$, $\deg_G(v) \geq \deg_Q(x)$ and v satisfies all constant/1-WL predicates defined on x in X of φ , where $\deg$ is the degree in G/Q ; in particular, $C_\varphi(x_0) = \{u\}$.

(3) *Refinement.* We iteratively refine C_φ via *DAG-graph dynamic programming*, alternating between *forward pass* on Q_D (top-down) and *backward pass* on its reverse Q_D^{-1} (bottom-up), until convergence. Each pass ensures that every vertex in C_φ satisfies both topological and predicate constraints (see [2]).

Example 11: In Example 10, Q_D is itself a DAG; its reverse Q_D^{-1} (resp. initial C_φ) are shown in Figure 7(b) (resp. 7(c)). After refinement, v_6 and v_7 are removed from $C_\varphi(x_2)$ since they do not have any order neighbor and thus, fail to match pattern edge (x_1, x_2) . Finally, we obtain final C_φ (Figure 7(d)). □

The candidate space C_φ has the following property.

Proposition 2: *Given pivot u and rule φ , DAG-graph dynamic programming computes a sound C_φ in $O(|Q||G|)$ time, i.e., for each witness h of φ with $h(x) = v$, v must be in $C_\varphi(x)$. □*

Proof. (1) DAG-graph dynamic programming is dominated by

pattern matching, which takes $O(|Q||G|)$ time [91]. (2) The soundness of C_φ is warranted as follows: (a) the initial C_φ is sound; (b) by induction, an auxiliary set $C'_\varphi(x)$ is computed correctly; and (c) if a vertex v does not satisfy the soundness condition, C_φ is still sound after we remove v from $C_\varphi(x)$. $\square$

B. Algorithm for Counterfactual Explanations

We develop an algorithm, CFE, to identify perturbations.

Intuition. Building on the candidate spaces of RxGNNs, algorithm CFE selects graph perturbations to render each candidate space C_φ *invalid*. As removing one vertex from C_φ triggers refinement and can cascade into further removals, the challenge is to minimize perturbations. To this end, CFE employs a *cost-effectiveness* strategy to decide which vertices to remove.

Below we first present the perturbation strategy, followed by the cost-effectiveness criterion. Finally we give the algorithm.

Perturbation strategy. Given $v \in C_\varphi(x)$, there are two types of perturbations that make v no longer map to x in *any* witness.

Feature modification. A GNN typically aggregates more information from *hub vertices* (i.e., with degrees above a threshold τ). If v is a hub vertex, we modify some important attributes A of v to cease to satisfy some predicates involving $x.A$ in X .

To measure attribute importance, we extend the GINI value adopted in [85], [58] to measure how well $x.A$ divides the pivots into different predicted classes of $\mathcal{M}$. Moreover, following [67], feature modification must be *plausible* (i.e., make sense in reality) and *feasible* (e.g., the gender is immutable). More details for feature modification are presented in [21].

Edge removal. If v is not a hub vertex, we remove incident edges of v so that all witnesses h with $h(x) = v$ no longer match Q . Assume w.l.o.g. that (x_p, l, x) is a selected pattern edge. We remove all edges (v_p, l, v) in G where $v_p \in C_\varphi(x_p)$.

Cost-effectiveness. To decide which variable x and vertex $v \in C_\varphi(x)$ to perturb, we consider both the *cost* and *effectiveness*.

Cost. According to the two perturbation strategies above, the cost of applying a perturbation to vertex v in $C_\varphi(x)$ is:

$$\text{cost}(v) = \begin{cases} \max_{x.A \text{ in } X} \text{imp}(x.A), & \text{if } X \neq \emptyset \text{ and } \deg_G(v) \geq \tau, \\ \beta |\{(v_p, l, v) \mid v_p \in C_\varphi(x_p)\}|, & \text{otherwise,} \end{cases}$$

where $\text{imp}(x.A)$ is the GINI-based importance score computed for $x.A$, τ is a degree threshold and β is a balancing parameter.

Effectiveness. Removing a vertex v from $C_\varphi(x)$ may trigger cascading removals in other sets $C_\varphi(x')$ ($x \neq x'$) during refinement. Thus, we measure the effectiveness of removing v on the entire candidate space C_φ , rather than only on the set $C_\varphi(x)$. More specifically, we compare the sizes of $C_\varphi(x)$ for all $x \in \bar{x}$ before and after refinement triggered by removing v .

Formally, we define the *effectiveness* of removing v as:

$$\text{eff}(v) = \alpha^K \left(1 - \min_{x \in \bar{x}} \frac{|C_\varphi^\Delta(x)|}{|C_\varphi(x)|}\right),$$

where vertex v is a K -hop neighbor of the pivot u , α is the hop-decay factor, and C_φ^Δ is the refined set C_φ after v is removed. Intuitively, vertices closer to u are more important

Input: A graph G , a model $\mathcal{M}$, a set Σ of RxGNNs and a vertex u .
Output: A set $\mathbb{O}$ of perturbations.

```

1.  $\mathbb{O} := \emptyset$ ;
2. for each RxGNN  $\varphi = Q[\bar{x}, x_0](X \rightarrow \neg \mathcal{M}(x_0))$  in  $\Sigma$  do
3.    $C_\varphi := \text{constructCand}(u, \varphi, G)$ ;
4.   while  $C_\varphi$  is valid do
5.      $v^* := \text{pickVertex}(C_\varphi)$ ; // pick the vertex to be removed
6.      $C_\varphi := \text{refine}(C_\varphi, v^*)$ ; // compute the refined  $C_\varphi$  after removing  $v^*$ 
7.      $\mathbb{O} = \mathbb{O} \cup \{\text{the perturbation that removes } v^* \text{ from } C_\varphi\}$ ;
8.   return  $\mathbb{O}$ ;
```

Fig. 8: Counterfactual algorithm CFE

and thus should be removed with higher priority; the smaller a set $C_\varphi^\Delta(x)$ is, the better the effectiveness is. Taken together, the *cost-effectiveness* of each $v \in C_\varphi$ is $\frac{\text{eff}(v)}{\text{cost}(v)}$.

Algorithm. We present algorithm CFE in Figure 8. It returns a set $\mathbb{O}$ such that for each φ in Σ , C_φ of φ is *invalid* on $G \ominus \mathbb{O}$.

We initialize $\mathbb{O} = \emptyset$ (line 1) and process each $\varphi \in \Sigma$ (lines 2–7). For each φ , we construct its C_φ pivoted at u via procedure `constructCand` (line 3). While C_φ remains valid (line 4), i.e., no $x \in \bar{x}$ has $C_\varphi(x) = \emptyset$, we select a vertex v^* to remove using procedure `pickVertex` based on cost-effectiveness (line 5, see below). We then remove v^* , refine C_φ (line 6), and add the corresponding perturbations to $\mathbb{O}$ (line 7). Once all C_φ 's are invalid, we return the set $\mathbb{O}$ of perturbations (line 8).

Example 12: Continuing with final C_φ in Example 11, if we remove v_4 , the resulting C_φ (i.e., $C_\varphi(x_0) = \{v_1\}$, $C_\varphi(x_1) = \{v_3\}$, $C_\varphi(x_2) = \{v_5\}$, $C_\varphi(x_3) = \{v_0\}$) is still valid, leading to additional perturbations to invalidate C_φ . Instead, it is more effective to remove v_5 since it directly invalidates C_φ (i.e., $C_\varphi(x_2) = \emptyset$) and we can finish the processing of φ . $\square$

Procedure pickVertex. To efficiently decide the most cost-effective vertex to remove, we cluster vertices in C_φ into groups. Intuitively, vertices in the same group share similar topology and thus likely induce similar effects. For each group g , we evaluate the cost-effectiveness of its center vertex v_g .

More specifically, let g^* be the group whose center vertex v_{g^*} achieves the highest cost-effectiveness. For each $v \in g^*$, we then compute its exact $\text{eff}(v)$ and $\text{cost}(v)$, and select the vertex v^* from g^* with the highest cost-effectiveness.

Complexity. The time complexity of CFE is $O(|\Sigma||C_\varphi||Q||G|(\#\text{group} + |g_{\max}|))$, where $\#\text{group}$ is the number of groups, and $g_{\max}$ is the largest group, since for each φ in Σ , (a) C_φ is computed in $O(|Q||G|)$ time, and (b) $O(|C_\varphi|)$ vertices will be removed from C_φ , where each removal takes $O(\#\text{group} + |g_{\max}|)$ times of refinement to decide.

VI. EXPERIMENTAL STUDY

Using real-life graphs, we empirically tested the accuracy and efficiency of local and global explanations of NEX.

Experimental setting. We start with the experimental setting.

Datasets. We used four real-life graphs G shown in Table II (cf. [2]): (1) Loan [92], a loan graph used to predict whether a user should be approved for loan; (2) Claim [93], a medical

Dataset	Loan	Claim	Trans	Amazon
$ V $	504K	702K	15.4M	1.7M
$ E $	1008K	1674K	12.7M	2.5M
#attr	13	16	10	15
#vertex labels	5	4	2	3
#classes	2	2	2	5

TABLE II: Dataset statistic (more in [2])

claim network dataset designed for insurance fraud detection; (3) Trans [94], a transactional network for financial fraud detection; and (4) Amazon [8], a multi-class e-commerce graph, where products are labeled with various classes.

GNNs. Following [95], [86], [96], we selected four GNN classifiers on heterogeneous graphs: (1) GCN [80], a standard message-passing graph convolutional network; (2) GAT [75], which uses attention mechanism; (3) GIN [6], which effectively captures graph structural information; and (4) HGT [76], which designs a vertex- and edge-type dependent attention mechanism. Unless stated explicitly, we used GCN by default.

Baselines. We compared two baselines for hybrid local and global explanations: (1) GVEX [48] and (2) CFGraph [49].

For local explanations, we tested three more baselines: (3) CFExp (*i.e.*, CF-GNNExplainer) [34], (4) CLEAR [37], and (5) InduCE [36]. For global explanations, we additionally tested: (6) XGNN [43], (7) GNNInt (*i.e.*, GNNInterpreter [44]), and (8) LMiner, which mines RxGNNs via traditional levelwise search; we only compared LMiner for efficiency.

We adapted all baselines to vertex classification on heterogeneous graphs and optimized all counterfactual methods to minimize perturbations (see more details in [2]).

Rules. On average, we mined 194, 357, 168 and 562 RxGNNs for 4 GNNs on Loan, Claim, Trans and Amazon, respectively.

Default parameters. We set the support threshold σ (resp. confidence δ) as 3K, 8K, 1K and 1K (resp. 0.7, 0.6, 0.7 and 0.5) on Loan, Claim, Trans and Amazon, respectively. We set (a) $k = 5$ (resp. $N = 15$) for the max size of motifs (resp. patterns), (b) $\eta = 0.6$ for motif identification, (c) $\alpha = 0.8$ for hop-decay, (d) $\beta = 0.01$ for the balancing parameter, and (e) $n = 8$ for the number of processors. We adopted GASTON [9] (resp. Qwen2.5 [20]) for pattern mining (resp. predicate ordering).

Environment. Experiments were conducted on a machine powered by 256GB RAM, 32 processors with Intel(R) Xeon(R) Gold 5320 CPU @2.20GHz and a NVIDIA Tesla V100 GPU with 32 GB memory. Each experiment was run 3 times, and the average is reported here. For limited space, representative results are reported on specific datasets/models.

Metrics. We used three widely-adopted metrics [86], [96], [95], [58] for local explanations. Each metric was evaluated on m randomly selected testing vertices. We set $m = 1000$.

(1) **Fidelity.** Since a counterfactual perturbs the input graph for a prediction switch at u , the counterfactual fidelity measures the ratio of successful prediction switches, defined as follows:

$$\text{fidel} = \frac{1}{m} \sum_u \mathbb{1}(\hat{y}_u \neq y_u),$$

where $\hat{y}_u$ (resp. y_u) is the model prediction on the perturbed $G \ominus \odot$ (resp. the original G), and $\mathbb{1}(\cdot)$ returns 1 if $\hat{y}_u \neq y_u$.

(2) **Sparsity.** It measures how minimal the edge perturbations are to swap negative predictions, defined as:

$$\text{spars} = \frac{1}{m} \sum_u \frac{\#\text{ptb_edges}_u}{\#\text{edges}_u},$$

where $\#\text{ptb_edges}_u$ (resp. $\#\text{edges}_u$) is the number of perturbed edges (resp. edges in L -hop neighborhood graph) of u .

(3) **Feature ratio.** We report the feature ratio, *i.e.*, the average ratio of attributes perturbed in the L -hop neighborhood graph.

For global explanations, we also adopt two widely used metrics [43], [45]: (1) **Overall recognizability**, measuring how well a GNN model $\mathcal{M}$ can recognize its own behavior based on the global explanations $\mathcal{E}$ (*e.g.*, $\mathcal{E} = \Sigma$ in NEX), *i.e.*,

$$\text{recog} = \frac{1}{|\mathcal{E}|} \sum_{\text{exp} \in \mathcal{E}} \mathbb{1}(\hat{y}(\text{exp}) = \text{false}),$$

where $\text{exp} \in \mathcal{E}$ is an explanation (*e.g.*, a canonical graph derived from each $\varphi \in \Sigma$), and $\hat{y}(\text{exp})$ is the prediction of $\mathcal{M}$ when exp is fed into it. (2) **Reliability**, evaluating the extent to which the global explanations $\mathcal{E}$ cover the testing vertices:

$$\text{relia} = \frac{|\bigcup_{\text{exp} \in \mathcal{E}} \text{testVertices}(\text{exp})|}{\#\text{totalVertices}},$$

where $\#\text{totalVertices}$ is the total number of testing vertices, and $\text{testVertices}(\text{exp})$ is the set of testing vertices explained by exp (*e.g.*, a vertex v is explained by φ if φ is applicable at v).

Experimental results. We next report our findings.

Exp-1: Local effectiveness. We tested effect of CFE in NEX.

Effectiveness. In Table III, on average, the fidelity of NEX is 8.30%, 60.85%, 53.71%, 24.98% and 20.17% higher than GVEX, CFGraph, CFExp, CLEAR and InduCE on the four datasets, respectively, up to 16.91%, 105.95%, 92.38%, 43.19% and 45.29%, since NEX finds necessary updates based on cost-effectiveness while the baselines depend heavily on hyper-parameters. Although GVEX also achieves reasonable fidelity, it is far less efficient (see Exp-2). The sparsity of NEX outperforms all baselines by $3.25\times$ on average, up to $7.00\times$.

For feature ratios (Figure 9(a)), NEX perturbs only 1.84% of features on average. In contrast, the five baselines ignore features altogether or apply a uniform feature mask across vertices, treating features across vertices identically.

Varying β . We varied β that controls the perturbation (*i.e.*, modify features or remove edges), from 0.001 to 1. As shown in Figure 9(b), when β gets larger, NEX tends to invalidate C_φ by modifying features only, leading to a lower sparsity and a higher feature ratio, as expected. In contrast, the fidelity of NEX slightly fluctuates and achieves the highest when $\beta = 0.01$ (not shown), where it strikes a balance.

Exp-2: Local efficiency. We evaluated the efficiency (*i.e.*, the average time for each testing vertex) of CFE in NEX.

Efficiency. In Table III, GVEX is slow since it relies on Jacobian computations to get influential subgraphs, unlike other methods that only select edges. By contrast, NEX is $7.30\times$ faster than other baselines; *e.g.*, on Trans it averages just 1.38s. This efficiency comes from using C_φ to avoid exhaustive enumeration and selecting perturbations via cost-effectiveness.

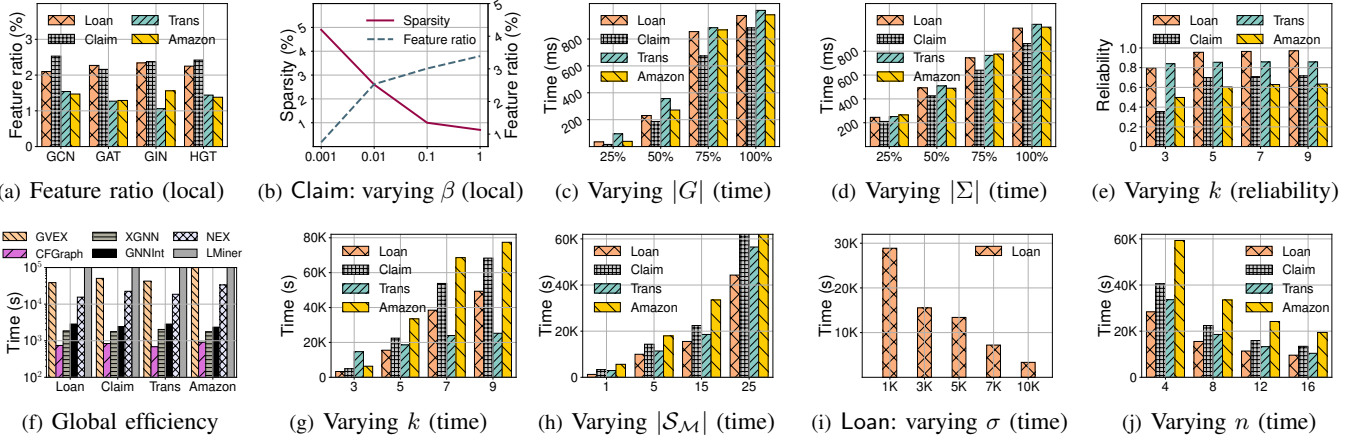


Fig. 9: Performance evaluation

GNN	Method	Loan			Claim			Trans			Amazon		
		fidel	spars	time(s)	fidel	spars	time(s)	fidel	spars	time(s)	fidel	spars	time(s)
GCN	GVEX	0.836	5.6%	367.3	0.615	4.3%	492.7	0.754	3.9%	432.3	0.693	5.3%	2587.5
	CFGraph	0.454	17.1%	3.45	0.441	15.5%	4.22	0.481	10.8%	3.30	0.655	10.2%	2.13
	CFExp	0.486	23.1%	1.66	0.383	14.7%	2.75	0.56	8.1%	1.99	0.683	11.1%	1.82
	Clear	0.653	10.2%	10.77	0.517	13.4%	15.54	0.635	7.1%	10.11	0.679	9.7%	24.37
	InduCE	0.687	13.7%	5.31	0.573	8.9%	6.81	0.703	5.3%	4.52	0.684	8.4%	4.69
	NEX	0.935	3.3%	0.97	0.719	2.6%	0.74	0.809	2.2%	1.36	0.734	3.3%	1.19
GAT	GVEX	0.843	5.3%	355.5	0.618	4.5%	485.7	0.748	3.9%	444.3	0.695	4.9%	2629.1
	CFGraph	0.456	18.2%	3.67	0.437	15.9%	4.91	0.466	12.3%	3.16	0.632	10.2%	2.73
	CFExp	0.510	18.4%	2.34	0.406	14.5%	3.52	0.555	9.7%	2.58	0.688	10.4%	1.89
	Clear	0.739	10.8%	14.44	0.526	14.7%	20.41	0.674	6.3%	12.48	0.689	9.1%	28.84
	InduCE	0.706	9.7%	5.57	0.619	8.2%	8.74	0.671	5.5%	4.42	0.672	8.1%	4.40
	NEX	0.936	2.9%	0.87	0.694	3.5%	0.84	0.806	3.1%	1.51	0.729	2.9%	1.34
GIN	GVEX	0.855	5.6%	403.4	0.626	4.5%	440.8	0.726	4.1%	499.9	0.685	5.5%	2632.4
	CFGraph	0.462	17.7%	4.25	0.455	14.4%	5.15	0.459	10.8%	4.50	0.651	9.9%	2.17
	CFExp	0.508	20.4%	2.57	0.375	16.2%	2.79	0.538	9.7%	2.40	0.680	10.6%	1.93
	Clear	0.663	15.3%	12.93	0.509	10.9%	17.23	0.641	5.4%	11.98	0.676	10.2%	22.68
	InduCE	0.638	13.4%	6.14	0.595	9.6%	8.86	0.629	5.8%	4.95	0.677	8.8%	4.29
	NEX	0.927	3.5%	1.00	0.690	3.1%	0.80	0.764	3.5%	1.29	0.715	3.8%	1.14
HGT	GVEX	0.839	5.1%	588.3	0.611	4.8%	695.6	0.735	3.9%	601.8	0.688	5.3%	3938.9
	CFGraph	0.453	15.6%	5.34	0.449	16.2%	6.20	0.469	13.3%	5.55	0.643	11.5%	3.52
	CFExp	0.505	20.5%	2.88	0.405	16.6%	5.22	0.542	9.2%	3.56	0.692	12.3%	2.15
	Clear	0.684	14%	14.43	0.512	10.5%	19.14	0.659	6.8%	14.71	0.695	9.2%	34.98
	InduCE	0.671	12.2%	6.99	0.602	9.2%	10.94	0.685	5.9%	6.27	0.684	9.1%	5.61
	NEX	0.929	3.5%	0.92	0.693	3.3%	0.96	0.797	3.3%	1.36	0.733	3.5%	1.38

TABLE III: Local (counterfactual) explanations

Varying $|G|$. We varied the scaling factor of G in Figure 9(c). It takes longer for NEX to handle larger graphs, *e.g.*, on Trans, NEX is $2.84\times$ slower when G changes from 50% to 100%.

Varying $|\Sigma|$. We varied the size of Σ in Figure 9(d). As expected, NEX takes longer with more RxGNNs, yet it remains efficient, handling hundreds of RxGNNs within about 1s.

Exp-3: Global effectiveness. We evaluated the rule set Σ returned by RxGNNMiner, which serves as global explanations.

Effectiveness. In Table IV, the average recog (resp. relia) of NEX is 15.65%, 167.21%, 80.42% and 17.34% (resp. 31.93%, 253.18%, 302.15% and 171.47%) higher than GVEX, CFGraph, XGNN and GNNInt, respectively, since rule discovery finds discriminative motifs that are mostly responsible for the predictions, while the baselines may produce patterns that are hardly satisfied in reality. In particular, the recog of NEX is 1 on Loan, since the prediction of loan application is highly individual-related, exhibiting clear regional characteristic (*e.g.*, low salary or lack of experience). All GNNs have learned these suspicious features, which also appear in nearly all RxGNNs.

Varying k . We varied the maximum size k of motifs from

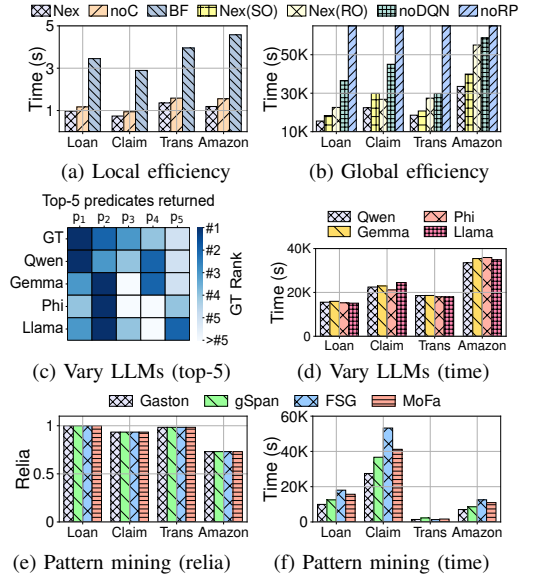


Fig. 10: Ablation study

3 to 9. As shown in Figure 9(e), for $k = 3$, the reliability of NEX on Claim is only 0.355, since the motifs are not sufficiently discriminative. Reliability improves as k increases, with optimal performance observed at $k = 5$.

Exp-4: Global efficiency. We tested runtime of RxGNNMiner.

Efficiency. As shown in Figure 9(f), NEX is at least $2.25\times$ faster than GVEX (which times out on Amazon), though not the fastest overall since it has to identify patterns, features and their conditions. In contrast, the baselines generate explanations without attributes or predicates. Note that the rule discovery in NEX incurs only a one-time offline cost.

Compared with the levelwise LMiner, which fails to finish within 24 hours, NEX mines rules efficiently through motif identification and composition. Although Trans is the largest dataset, it is not the slowest for NEX, as its limited vertex label types (Table II) yield simpler motif combinations.

Varying k . We tested the efficiency of NEX by varying k (Figure 9(g)). As k increases, more k -bounded motifs are generated, leading to longer runtime; *e.g.*, increasing k from 3 to 9 makes rule discovery $10.64\times$ slower on average. On

GNN	Method	Loan		Claim		Trans		Amazon	
		recog	relia	recog	relia	recog	relia	recog	relia
GCN	GVEX	0.763	0.652	0.843	0.575	0.687	0.672	0.691	0.502
	CFGGraph	0.409	0.221	0.276	0.158	0.313	0.496	0.426	0.145
	XGNN	0.373	0.167	0.656	0.174	0.528	0.353	0.418	0.183
	GNNInt	0.755	0.545	0.783	0.218	0.691	0.475	0.697	0.221
	NEX	1.000	0.956	0.934	0.699	0.984	0.855	0.731	0.604
GAT	GVEX	0.768	0.607	0.857	0.551	0.698	0.688	0.663	0.486
	CFGGraph	0.384	0.229	0.255	0.174	0.299	0.510	0.445	0.175
	XGNN	0.414	0.143	0.613	0.176	0.535	0.316	0.470	0.191
	GNNInt	0.735	0.511	0.809	0.113	0.737	0.458	0.719	0.245
	NEX	1.000	0.961	0.967	0.702	0.971	0.851	0.745	0.601
GIN	GVEX	0.807	0.616	0.833	0.549	0.696	0.654	0.682	0.508
	CFGGraph	0.432	0.197	0.273	0.144	0.255	0.539	0.476	0.184
	XGNN	0.410	0.182	0.595	0.154	0.458	0.320	0.459	0.138
	GNNInt	0.787	0.437	0.771	0.166	0.677	0.305	0.688	0.267
	NEX	1.000	0.955	0.953	0.658	1.000	0.887	0.744	0.614
HGT	GVEX	0.776	0.634	0.861	0.573	0.678	0.694	0.661	0.455
	CFGGraph	0.434	0.178	0.212	0.197	0.260	0.474	0.424	0.177
	XGNN	0.398	0.192	0.587	0.160	0.503	0.298	0.429	0.180
	GNNInt	0.765	0.507	0.797	0.188	0.662	0.437	0.694	0.233
	NEX	1.000	0.944	0.932	0.691	0.965	0.872	0.728	0.617

TABLE IV: Global effectiveness

Trans, however, the growth is smoother, as fewer label types allow smaller k to already cover most frequent patterns.

Varying #motifs in $\mathcal{S}_M$ gets similar trend (Figure 9(h)).

Varying σ or δ . We varied support σ from 1K to 10K on Loan in Figure 9(i). As expected, when σ increases, NEX runs faster by filtering more rules by anti-monotonicity. However, since confidence is not anti-monotonic, it gets slightly slower (not shown) since more rules are checked when δ increases.

Varying n . Varying the number n of processors, we tested the parallel scalability of RxGNNMiner in Figure 9(j). It is $3.06\times$ faster when n varies from 4 to 16, *i.e.*, it is parallelly scalable.

Exp-5: Ablation study. We tested the variants of NEX, by disabling/varying some components while keeping the rest intact.

- (1) For local explanations, we tested two variants (Fig. 10(a)).
 - *Without clustering.* We compared noC, that computes cost-effectiveness for all vertices. NEX beats noC by $1.24\times$.
 - *Without candidate space.* Compared to brute-force method BF, which enumerates all witnesses, NEX is $3.55\times$ faster.
- (2) For global efficiency, we disable the following (Fig. 10(b)).
 - *Without DQN.* We compared a variant noDQN of NEX which composes motifs without the guidance of DQN. DQN accelerates rule discovery by $1.93\times$ on average.
 - *Without restricted predicate space (RP).* We tested noRP that exhaustively generates all possible predicates for each given pattern. This variant cannot finish in 24 hours, verifying the benefit of restricting the predicate space.
- (3) We further evaluated the following components in NEX.

Varying predicate orders. To test the impact of our perplexity predicate order (PPO) on the rule discovery time, we replaced it with: (a) a descending order of supports [7] (SO) and (b) a random order (RO) in Figure 10(b). All variants only differ in efficiency. NEX with PPO is $1.21\times$ and $1.44\times$ faster than the variants using SO and RO, respectively, since it improves multi-thread parallelism by prioritizing useful predicates. SO is even slower than RO on Claim since processing high-support predicates may delay pruning (see a case study in [2]).

Varying LLMs. We tested the impact of LLMs on rule discovery time and resulting predicate ordering using Qwen2.5 [20], Gemma3 [21], Phi3 [23] and Llama3.2 [22]. All LLMs produce reasonable orderings, yielding similar discovery times (Figure 10(d)), but differ in ordering accuracy, *e.g.*, using a ground-truth (GT) order derived from predicate frequencies in final valid rules (see [2]), we compared each LLM’s top-5 predicates on Claim via the heatmap in Figure 10(c). Qwen2.5 best matches the GT order, recovering all GT top-5 predicates and misordering only one (p_4); we thus adopt it as the default.

Varying pattern mining methods. We evaluated the time and results of pattern mining, by comparing GASTON with other methods, including gSpan [10], FSG [11] and MoFa [12]. Since all methods run exact mining given same inputs and mining requirements, they produce identical patterns and thus, the same relia (Figure 10(e)). However, GASTON runs the fastest (Figure 10(f)), except on dataset Trans, which contains only a small number of patterns (see Exp-4), resulting in fast mining time for all methods. Since GASTON achieves good efficiency without compromising accuracy, we used it as our default.

(4) For global effectiveness, we compared the following.

- *Without general patterns.* We tested NEX with star-pattern rules (*i.e.*, an adaptation of [58], [84], [85]). This restriction reduces expressivity: on Amazon, recog of NEX falls from 0.731 to 0.427 and relia from 0.604 to 0.224 (not shown), because star patterns cannot capture sophisticated dependencies beyond hub-and-spoke structures, among others.
- *Without 1-WL predicates.* The expressivity of RxGNNs without 1-WL predicates is also greatly reduced, *e.g.*, on Amazon, recog of NEX falls from 0.731 to 0.537 and relia from 0.604 to 0.417, justifying the need of 1-WL tests.

Summary. We find the following. (1) *Effective local counterfactuals:* Across four real-world graphs, NEX improves fidelity by 33.60% and sparsity by $3.25\times$ over prior methods. (2) *Efficiency:* NEX generates a counterfactual in only 1.38s on average, even for graphs with 15.4M vertices and 12.7M edges. (3) *Accurate global explanations:* The discovered RxGNNs achieve 70.16% higher recognizability and 189.68% higher reliability on average, with gains up to 339.62% and 572.03% over baselines. (4) *Scalability:* Its discovery algorithm is parallelly scalable, enabling efficient rule mining on large graphs.

VII. CONCLUSION

NEX makes several contributions for explaining GNN-based classifiers: (a) a unified logical framework that provides both global and local explanations for negative predictions; (b) RxGNNs, a new class of graph rules that incorporate negated predicates and the 1-WL test to capture diverse GNN behaviors with higher expressiveness; (c) a model-guided, parallelly scalable algorithm for efficient discovery of RxGNNs as global explanations; and (d) a cost-effective vertex-filtering algorithm for generating local counterfactual explanations. Empirical results confirm that NEX is promising in practice.

Future work includes extending NEX beyond GNN models to general models, for factual and counterfactual explanations.

VIII. AI-GENERATED CONTENT ACKNOWLEDGMENT

The content (including but not limited to text, figures, images, and code) was not generated by artificial intelligence.

REFERENCES

- [1] “Code and dataset,” 2025, <https://github.com/hdqlkncso24jvf/NEX>.
- [2] “Full version,” 2025, https://github.com/hdqlkncso24jvf/NEX/blob/main/paper_full_version.pdf.
- [3] Y. Hu, X. Wang, Z. Lin, P. Li, and M. Zhang, “Two-dimensional weisfeiler-lehman graph neural networks for link prediction,” *CoRR*, vol. abs/2206.09567, 2022.
- [4] M. Grohe, “The logic of graph neural networks,” in *LICS*, 2021.
- [5] C. Morris, M. Ritzert, M. Fey, W. L. Hamilton, G. Rattan, and M. Grohe, “Weisfeiler and leman go neural: Higher-order graph neural networks,” in *AAAI*, 2019, pp. 4602–4609.
- [6] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *ICLR*, 2019.
- [7] W. Fan, Z. Han, Y. Wang, and M. Xie, “Discovering top-k rules using subjective and objective criteria,” in *SIGMOD*. ACM, 2023.
- [8] “Amazon product graph,” 2025, <https://cseweb.ucsd.edu/~jmcauley/datasets/>.
- [9] S. Nijssen and J. N. Kok, “The Gaston tool for frequent subgraph mining,” *Electronic Notes in Theoretical Computer Science*, vol. 127, no. 1, pp. 77–87, 2005.
- [10] X. Yan and J. Han, “gSpan: Graph-based substructure pattern mining,” in *International Conference on Data Mining*. IEEE, 2002.
- [11] M. Kuramochi and G. Karypis, “Frequent subgraph discovery,” in *IEEE international conference on data mining*. IEEE, 2001, pp. 313–320.
- [12] C. Borgelt and M. R. Berthold, “Mining molecular fragments: Finding relevant substructures of molecules,” in *2002 IEEE International Conference on Data Mining*. IEEE, 2002, pp. 51–58.
- [13] X. Chu, I. F. Ilyas, and P. Papotti, “Discovering denial constraints,” *PVLDB*, vol. 6, no. 13, pp. 1498–1509, 2013.
- [14] E. H. M. Pena, E. C. de Almeida, and F. Naumann, “Discovery of approximate (and exact) denial constraints,” *PVLDB*, vol. 13, no. 3, pp. 266–278, 2019.
- [15] W. Fan, C. Hu, X. Liu, and P. Lu, “Discovering graph functional dependencies,” *TODS*, vol. 45, no. 3, pp. 1–42, 2020.
- [16] W. Fan, W. Fu, R. Jin, P. Lu, and C. Tian, “Discovering association rules from big graphs,” *PVLDB*, vol. 15, no. 7, pp. 1479–1492, 2022.
- [17] H. Chen, W. Fan, and J. Zheng, “Incremental rule discovery in response to parameter updates,” *Proc. ACM Manag. Data*, vol. 3, no. 3, pp. 175:1–175:28, 2025.
- [18] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, “The case for learned index structures,” in *SIGMOD*, 2018, pp. 489–504.
- [19] H. Mao, M. Schwarzkopf, S. B. Venkatarishnan, Z. Meng, and M. Alizadeh, “Learning scheduling algorithms for data processing clusters,” in *SIGCOMM*, 2019, pp. 270–288.
- [20] Qwen team, “Qwen2.5 technical report,” 2025. [Online]. Available: <https://arxiv.org/abs/2412.15115>
- [21] Gemma team, “Gemma 3 technical report,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.19786>
- [22] Llama 3 team, “The llama 3 herd of models,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [23] Phi-3 team, “Phi-3 technical report: A highly capable language model locally on your phone,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.14219>
- [24] C. Gao, Y. Zheng, N. Li, Y. Li, Y. Qin, J. Piao, Y. Quan, J. Chang, D. Jin, X. He, and Y. Li, “A survey of graph neural networks for recommender systems: Challenges, methods, and directions,” *ACM Transactions on Recommender Systems*, vol. 1, no. 1, pp. 1–51, 2023.
- [25] S. Wu, W. Zhang, F. Sun, and B. Cui, “Graph neural networks in recommender systems: A survey,” *CoRR*, vol. abs/2011.02260, 2020.
- [26] S. X. Rao, S. Zhang, Z. Han, Z. Zhang, W. Min, Z. Chen, Y. Shan, Y. Zhao, and C. Zhang, “xFraud: Explainable fraud transaction detection,” *PVLDB*, vol. 15, no. 3, pp. 427–436, 2021.
- [27] Y. Luo, G. Wang, Y. Liu, J. Yue, W. Cheng, and B. Fei, “FAF: A risk detection framework on industry-scale graphs,” in *CIKM*, 2023, pp. 4717–4723.
- [28] M. Sun, S. Zhao, C. Gilvary, O. Elemento, J. Zhou, and F. Wang, “Graph convolutional networks for computational drug development and discovery,” *Briefings in bioinformatics*, vol. 21, no. 3, 2020.
- [29] J. Xiong, Z. Xiong, K. Chen, H. Jiang, and M. Zheng, “Graph neural networks for automated de novo drug design,” *Drug discovery today*, vol. 26, no. 6, pp. 1382–1393, 2021.
- [30] Y. L. Liu, Y. Wang, O. Vu, R. Moretti, B. Bodenheimer, J. Meiler, and T. Derr, “Interpretable chirality-aware graph neural network for quantitative structure activity relationship modeling in drug discovery,” in *AAAI*, vol. 37, no. 12, 2023, pp. 14 356–14 364.
- [31] D. Qiu, M. Wang, A. Khan, and Y. Wu, “Generating robust counterfactual witnesses for graph neural networks,” in *ICDE*. IEEE, 2024, pp. 3351–3363.
- [32] J. Cui, M. Yu, B. Jiang, A. Zhou, J. Wang, and W. Zhang, “Interpretable knowledge tracing via response influence-based counterfactual reasoning,” in *ICDE*, 2024.
- [33] S. An and Y. Cao, “Counterfactual explanation at will, with zero privacy leakage,” *PACMOD*, 2024.
- [34] A. Lucic, M. A. Ter Hoeve, G. Tolomei, M. De Rijke, and F. Silvestri, “CF-GNNExplainer: Counterfactual explanations for graph neural networks,” in *International Conference on Artificial Intelligence and Statistics*. PMLR, 2022, pp. 4499–4511.
- [35] M. Bajaj, L. Chu, Z. Y. Xue, J. Pei, L. Wang, P. C.-H. Lam, and Y. Zhang, “Robust counterfactual explanations on graph neural networks,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 5644–5655, 2021.
- [36] S. Verma, B. Armgaan, S. Medya, and S. Ranu, “InduCE: Inductive counterfactual explanations for graph neural networks,” *Transactions on Machine Learning Research*, 2024.
- [37] J. Ma, R. Guo, S. Mishra, A. Zhang, and J. Li, “CLEAR: Generative counterfactual explanations on graphs,” *Advances in neural information processing systems*, vol. 35, pp. 25 895–25 907, 2022.
- [38] H.-M. Attolou, K. Tzompanaki, K. Stefanidis, and D. Kotzinos, “Why-not explainable graph recommender,” in *ICDE*, 2024.
- [39] D. Qiu, J. Chen, A. Khan, Y. Zhao, and F. Bonchi, “Finding counterfactual evidences for node classification,” in *SIGKDD*, 2025.
- [40] J. Tan, S. Geng, Z. Fu, Y. Ge, S. Xu, Y. Li, and Y. Zhang, “Learning and evaluating graph neural network explanations based on counterfactual and factual reasoning,” in *The Web Conference*, 2022, pp. 1018–1027.
- [41] Z. Chen, F. Silvestri, J. Wang, Y. Zhang, Z. Huang, H. Ahn, and G. Tolomei, “GREASE: Generate factual and counterfactual explanations for GNN-based recommendations,” *CoRR*, vol. abs/2208.04222, 2022.
- [42] M. A. Prado-Romero, B. Prenkaj, G. Stilo, and F. Giannotti, “A survey on graph counterfactual explanations: Definitions, methods, evaluation, and research challenges,” *ACM Comput. Surv.*, vol. 56, no. 7, pp. 171:1–171:37, 2024.
- [43] H. Yuan, J. Tang, X. Hu, and S. Ji, “XGNN: Towards model-level explanations of graph neural networks,” in *SIGKDD*, 2020.
- [44] X. Wang and H. W. Shen, “GNNInterpreter: A probabilistic generative model-level explanation for graph neural networks,” in *International Conference on Learning Representations*, 2022.
- [45] G. Lv and L. Chen, “On data-aware global explainability of graph neural networks,” *PVLDB*, vol. 16, no. 11, pp. 3447–3460, 2023.
- [46] Z. Huang, M. Kosan, S. Medya, S. Ranu, and A. Singh, “Global counterfactual explainer for graph neural networks,” in *ACM International Conference on Web Search and Data Mining*, 2023, pp. 141–149.
- [47] G. Fournier and S. Medya, “COMRECGC: Global graph counterfactual explainer through common recourse,” *arXiv preprint arXiv:2505.07081*, 2025.
- [48] T. Chen, D. Qiu, Y. Wu, A. Khan, X. Ke, and Y. Gao, “View-based explanations for graph neural networks,” *Proc. ACM Manag. Data*, vol. 2, no. 1, pp. 1–27, 2024.
- [49] C. Abrate and F. Bonchi, “Counterfactual graphs for explainable classification of brain networks,” in *SIGKDD*, 2021, pp. 2495–2504.
- [50] “The dataset for job application prediction,” 2024, <https://www.kaggle.com/datasets/sumittr26/employee-upskilling-and-hiring-success-dataset>.
- [51] B. Y. Weisfieler and A. A. Leman, “The reduction of a graph to canonical form and the algebra which appears therein,” *NIT*, vol. 2, 1968.
- [52] H. L. Morgan, “The generation of a unique machine description for chemical structures—a technique developed at chemical abstracts service,” *Journal of Chemical Documentation*, vol. 5, no. 2, pp. 107–113, 2065.
- [53] J. Cai, M. Fürer, and N. Immerman, “An optimal lower bound on the number of variables for graph identification,” *Comb.*, vol. 12, no. 4, pp. 389–410, 1992.

- [54] M. Grohe, “word2vec, node2vec, graph2vec, x2vec: Towards a theory of vector embeddings of structured data,” in *PODS*, 2020, pp. 1–16.
- [55] C. P. Kruskal, L. Rudolph, and M. Snir, “A complexity theory of efficient parallel algorithms,” *Theor. Comput. Sci.*, vol. 71, no. 1, pp. 95–132, 1990.
- [56] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. A. Riedmiller, A. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nat.*, 2015.
- [57] S. Azzolin, A. Longa, P. Barbiero, P. Liò, and A. Passerini, “Global explainability of GNNs via logic combination of learned concepts,” in *ICLR 2023*, 2023, pp. 1–19.
- [58] W. Fan, L. Fan, D. Lin, and M. Xie, “Explaining GNN-based recommendations in logic,” *PVLDB*, vol. 18, no. 3, pp. 715–728, 2024.
- [59] C. Rudin and Y. Shaposhnik, “Globally-consistent rule-based summary-explanations for machine learning models: Application to credit-risk evaluation,” *J. Mach. Learn. Res.*, vol. 24, pp. 16:1–16:44, 2023.
- [60] C. Chen, K. Lin, C. Rudin, Y. Shaposhnik, S. Wang, and T. Wang, “An interpretable model with globally consistent explanations for credit risk,” *CoRR*, vol. abs/1811.12615, 2018.
- [61] Z. Geng, M. Schleich, and D. Suciu, “Computing rule-based explanations by leveraging counterfactuals,” *PVLDB*, vol. 16, no. 3, 2022.
- [62] L. Bertossi, “Declarative approaches to counterfactual explanations for classification,” *Theory and Practice of Logic Programming*, vol. 23, no. 3, pp. 559–593, 2023.
- [63] G. Audemard, J.-M. Lagniez, and P. Marquis, “On the computation of contrastive explanations for boosted regression trees,” in *The 27th European Conference on Artificial Intelligence*, 2024.
- [64] L. Bertossi, J. Li, M. Schleich, D. Suciu, and Z. Vagena, “Causality-based explanation of classification outcomes,” in *International Workshop on Data Management for End-to-End Machine Learning*, 2020, pp. 1–10.
- [65] J. Marques-Silva, “Logic-based explainability: Past, present and future,” in *International Symposium on Leveraging Applications of Formal Methods*. Springer, 2024, pp. 181–204.
- [66] A. Darwiche, “Logic for explainable AI,” in *Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. IEEE, 2023, pp. 1–11.
- [67] M. Schleich, Z. Geng, Y. Zhang, and D. Suciu, “GeCo: Quality counterfactual explanations in real time,” *PVLDB*, 2021.
- [68] P. M. VanNostrand, H. Zhang, D. M. Hofmann, and E. A. Rundensteiner, “FACET: Robust counterfactual explanation analytics,” *Proc. ACM Manag. Data*, vol. 1, no. 4, pp. 242:1–242:27, 2023.
- [69] A. Karimi, G. Barthe, B. Balle, and I. Valera, “Model-agnostic counterfactual explanations for consequential decisions,” *CoRR*, vol. abs/1905.11190, 2019.
- [70] R. K. Mothilal, A. Sharma, and C. Tan, “Explaining machine learning classifiers through diverse counterfactual explanations,” in *Conference on Fairness, Accountability, and Transparency (FAT)*. ACM, 2020, pp. 607–617.
- [71] W. Fan, Y. Wu, and J. Xu, “Functional dependencies for graphs,” in *SIGMOD*. ACM, 2016, pp. 1843–1857.
- [72] W. Fan, R. Jin, M. Liu, P. Lu, C. Tian, and J. Zhou, “Capturing associations in graphs,” *PVLDB*, vol. 13, no. 11, pp. 1863–1876, 2020.
- [73] W. Fan and P. Lu, “Dependencies for graphs,” *ACM Trans. Database Syst. (TODS)*, vol. 44, no. 2, pp. 5:1–5:40, 2019.
- [74] W. Akhtar, A. Cortés-Calabuig, and J. Paredaens, “Constraints in RDF,” in *SDKB*, 2010.
- [75] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations*, 2018.
- [76] Z. Hu, Y. Dong, K. Wang, and Y. Sun, “Heterogeneous graph transformer,” in *The Web conference*, 2020, pp. 2704–2710.
- [77] S. Yun, M. Jeong, R. Kim, J. Kang, and H. J. Kim, “Graph transformer networks,” *Advances in neural information processing systems*, 2019.
- [78] H. Tang, C. Wang, J. Zheng, and C. Jiang, “Enabling graph neural networks for semi-supervised risk prediction in online credit loan services,” *ACM Transactions on Intelligent Systems and Technology*, vol. 15, no. 1, pp. 1–24, 2024.
- [79] M. Zopf, “1-WL expressiveness is (almost) all you need,” in *International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2022, pp. 1–8.
- [80] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” *ICLR*, 2017.
- [81] D. Qiu, H. Che, A. Khan, and Y. Wu, “Generating skyline explanations for graph neural networks,” *arXiv preprint arXiv:2505.07635*, 2025.
- [82] W. Fan, X. Wang, Y. Wu, and J. Xu, “Association rules with graph patterns,” *PVLDB*, vol. 8, no. 12, pp. 1502–1513, 2015.
- [83] W. Fan, R. Jin, P. Lu, C. Tian, and R. Xu, “Towards event prediction in temporal graphs,” *PVLDB*, vol. 15, no. 9, pp. 1861–1874, 2022.
- [84] W. Fan, W. Fu, R. Jin, M. Liu, P. Lu, and C. Tian, “Making it tractable to catch duplicates and conflicts in graphs,” *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 86:1–86:28, 2023.
- [85] L. Fan, W. Fan, P. Lu, C. Tian, and Q. Yin, “Enriching recommendation models with logic conditions,” *Proc. ACM Manag. Data*, vol. 1, no. 3, pp. 210:1–210:28, 2024.
- [86] Z. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, “GN-Explainer: Generating explanations for graph neural networks,” in *NeurIPS*, 2019, pp. 9240–9251.
- [87] C. Jiang, F. Coenen, and M. Zito, “A survey of frequent subgraph mining algorithms,” *Knowledge Eng. Review*, vol. 28, no. 01, pp. 75–105, 2013.
- [88] Y. Huhtala, J. Kärkkäinen, P. Porkka, and H. Toivonen, “TANE: An efficient algorithm for discovering functional and approximate dependencies,” *The computer journal*, vol. 42, no. 2, 1999.
- [89] M. Garey and D. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, 1979.
- [90] N. Alon, R. Yuster, and U. Zwick, “Color-coding,” *Journal of the ACM (JACM)*, vol. 42, no. 4, pp. 844–856, 1995.
- [91] M. Han, H. Kim, G. Gu, K. Park, and W.-S. Han, “Efficient subgraph matching: Harmonizing dynamic programming, adaptive matching order, and failing set together,” in *SIGMOD*, 2019, pp. 1429–1446.
- [92] “The dataset for loan approval prediction,” 2024, <https://www.kaggle.com/datasets/hydracsnova/loan-approval-dataset>.
- [93] R. Zhang, D. Cheng, J. Yang, Y. Ouyang, X. Wu, Y. Zheng, and C. Jiang, “Pre-trained online contrastive learning for insurance fraud detection,” in *AAAI*, vol. 38, no. 20, 2024, pp. 22 511–22 519.
- [94] “The dataset for fraudulent activities detection,” 2024, <https://www.kaggle.com/datasets/rohit265/fraud-detection-dynamics-financial-transaction>.
- [95] H. Yuan, H. Yu, J. Wang, K. Li, and S. Ji, “On explainability of graph neural networks via subgraph explorations,” in *ICML*. PMLR, 2021, pp. 12 241–12 252.
- [96] H. Yuan, H. Yu, S. Gui, and S. Ji, “Explainability in graph neural networks: A taxonomic survey,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 45, no. 5, pp. 5782–5799, 2022.
- [97] K. Xu, C. Li, Y. Tian, T. Sonobe, K.-i. Kawarabayashi, and S. Jegelka, “Representation learning on graphs with jumping knowledge networks,” in *International conference on machine learning*. PMLR, 2018, pp. 5453–5462.
- [98] S. Zhang, H. Li, M. Wang, M. Liu, P.-Y. Chen, S. Lu, S. Liu, K. Murugesan, and S. Chaudhury, “On the convergence and sample complexity analysis of deep q-networks with ϵ -greedy exploration,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 13 064–13 102, 2023.
- [99] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Wadsworth, 1984.
- [100] M. V. Gwetu, J. Tapamo, and S. Viriri, “Random forests with a steepend Gini-index split function and feature coherence injection,” in *MLN*, 2019, pp. 255–272.
- [101] W. Fan, P. Lu, and C. Tian, “Unifying logic rules and machine learning for entity enhancing,” *Sci. China Inf. Sci.*, vol. 63, no. 7, 2020.
- [102] W. Fan, F. Geerts, X. Jia, and A. Kementsietsidis, “Conditional functional dependencies for capturing data inconsistencies,” *TODS*, vol. 33, no. 2, p. 6, 2008.
- [103] W. Fan, H. Gao, X. Jia, J. Li, and S. Ma, “Dynamic constraints for record matching,” *Vldb J.*, vol. 20, no. 4, pp. 495–520, 2011.
- [104] C. Wyss, C. Giannella, and E. Robertson, “FastFD: A heuristic-driven, depth-first algorithm for mining functional dependencies from relation instances extended abstract,” in *DaWaK*, 2001.
- [105] A. Cortés-Calabuig and J. Paredaens, “Semantics of constraints in RDFS,” in *AMW*, 2012.

APPENDIX

This appendix is organized as follows.

- Appendix A: Details about Rule Discovery
- Appendix B: Details of Local Counterfactual Explanation
- Appendix C: Other Related Work
- Appendix D: Details about Experiments

Appendix A: Details about Rule Discovery

This section provides additional details of our rule discovery approach, RxGNNMiner. We first describe the motif identification strategy, which mines σ -frequent and k -bounded motifs (Appendix A-1). Next, we elaborate on motif composition using DQN (Appendix A-2). We then discuss the parallel scalability of RxGNNMiner (Appendix A-3). Finally, we explain how to compute a cover of RxGNNs (Appendix A-4).

Appendix A-1: Discussion about Motif Identification

By our motif identification strategy, we mine only σ -frequent and k -bounded motifs, justified as follows.

(1) The use of σ is to prune useless motifs without missing any rule above the support threshold. Specifically, given a motif π and a pattern Q composed from π (possibly with other motifs), π subsumes Q and thus, $\text{supp}(\pi, G) \geq \text{supp}(Q, G)$ by anti-monotonicity. Therefore, if a motif is not σ -frequent, all patterns composed from it will not be σ -frequent, which can be safely excluded from consideration.

(2) We use k to balance cost and discriminability: if k is too small, the motifs may not be discriminative enough; and if k is too large, it becomes costly to obtain all k -bounded motifs. In practice, it often suffices to set k small (e.g., $k \leq 5$), e.g., the best performance of a GNN-based model is observed when it aggregates information from its 2-hop neighbors [97], and “deeper versions of the model that, in principle, have access to more information, perform worse [80]”.

Remark on parameter k . In practice, users can often (a) start with a conservative k (e.g., $k = 3$), incrementally increase it, and stop when marginal gains diminish; or (b) set time or memory budget, and dynamically adjust k to stay within resource limits; or (c) leverage data statistics to select a suitable k ; or (d) train ML models to guide the choice of k . In addition, incremental tuning [17] can also be employed for k .

Appendix A-2: Details of Motif Composition

Motif composition leads to a large state space. To address this, we adopt deep Q-learning (DQN) [56], which handles large state spaces efficiently and models complex structures with neural networks for better generalization. In contrast, traditional Q-learning, while simpler, struggles with scalability due to large Q-tables and inefficiency. Below we first formulate motif composition as a Markov Decision Process (MDP), and then describe the training and inference of DQN.

MDP. Given a set S of motifs, we create an initial pattern $Q_0[\bar{x}, x_0]$ by merging vertices x_0 from motifs in S into one x_0 in Q_0 . We model the composition as a sequential decision process: given the current $Q[\bar{x}, x_0]$, we make a decision by

selecting a pair of vertices in Q with the same label to merge. This can be modeled as an MDP in the following.

- *State:* The current pattern Q represents a state s .
- *Action:* Each mergable pair (v, v') of vertices in Q (i.e., $L_Q(v) = L_Q(v')$) is an action a . We also use a special action [end] to terminate the composition, and stop if the $\text{supp}(Q, G) < \sigma$ (by anti-monotonicity). Denote the action space of a state s by $\mathcal{A}(s)$.
- *Transition:* We transit from a state to another by taking an action, e.g., given the current state $s_i = Q_i$ and an action $a = (v, v')$, we transit to a new state $s_{i+1} = Q_{i+1}$, denoted as $s_i \rightarrow s_{i+1}$, where v and v' are represented as a single merged vertex in Q_{i+1} .
- *Reward:* A reward r is associated with the transition $s_i \rightarrow s_{i+1}$, indicating the effect of taking action (v, v') :

$$r = \begin{cases} -1 & \text{if } \text{supp}(Q_{i+1}, G) < \sigma, \\ \text{conf}(Q_{i+1}, G) - \text{conf}(Q_i, G) & \text{if } \text{supp}(Q_{i+1}, G) \geq \sigma. \end{cases}$$

Training of DQN. To utilize DQN for pattern composition, we adopt several component networks: (a) a *state encoder*, $\text{encoder}_\phi(Q)$ (resp. *action encoder*, $\text{encoder}_\phi(v, v')$), with parameter ϕ that maps a state $s = Q$ (resp. an action $a = (v, v')$) to an embedding $\mathbf{e}_s$ (resp. $\mathbf{e}_a$), and (b) a *Q-value network* $\mathbb{Q}_\theta(s, a)$ with parameter θ that predicts the expected accumulated reward (i.e., Q-value) by taking action a at s .

Specifically, we leverage the vertex embedding $\mathbf{e}_v^L$ outputted by the L -layer GNN model $\mathcal{M}$ to encode a pattern Q , i.e.,

$$\mathbf{e}_s = \text{encoder}_\phi(s) = \text{readout}(\{\mathbf{e}_v^L \mid v \in Q \text{ where } s = Q\}),$$

where the readout function aggregates vertex embeddings into a pattern-level representation; likewise, action embeddings are obtained by concatenating vertex embeddings, i.e.,

$$\mathbf{e}_a = \text{encoder}_\phi(a) = \text{concat}(\mathbf{e}_v^L, \mathbf{e}_{v'}^L), \text{ where } a = (v, v').$$

For the Q-value network, we adopt

$$\mathbb{Q}_\theta(s, a) = \text{FFN}_\theta(\mathbf{e}_s, \mathbf{e}_a),$$

where $\text{FFN}()$ is a feed-forward neural network. We train it with the ϵ -greedy strategy, minimizing the loss $\mathcal{L}(\theta, \phi)$:

$$\mathbb{E}_{(s_i, a, r, s_{i+1}) \sim \mathcal{T}} \left[\left(r + \gamma \max_{a' \in \mathcal{A}(s_{i+1})} \mathbb{Q}_{\theta'}(s_{i+1}, a') - \mathbb{Q}_\theta(s_i, a) \right)^2 \right],$$

where $\mathcal{T}$ is a replay buffer containing transitions (s_i, a, r, s_{i+1}) (i.e., we transit from s_i to s_{i+1} by taking action a with reward r), γ is a discount factor, and θ' is the parameter of a target network $\mathbb{Q}_{\theta'}$, periodically copied from $\mathbb{Q}_\theta$ for providing a stable target during training.

DQN Inference. Given the trained $\mathbb{Q}_\theta(s, a)$, we initialize with pattern Q_0 and iteratively merge the vertex pair with the highest Q-value until the termination condition is met.

Sample complexity. The sample complexity of DQN with ϵ -greedy is polynomially with the network size (e.g., linear to the dimension of the feature space for state-action pairs and the number of layers in the neural network), and inversely with the square of the desired accuracy (see [98]).

Appendix A-3: Extended Discussion of Discovery

This section extends discussion on RxGNNMiner.

Parallel scalability. We adopt *parallel scalability* [55] to measure the effectiveness of RxGNNMiner. Consider a problem $\mathbb{P}$ on graph G . We denote by $T_s(|I_{\mathbb{P}}|, |G|)$ the worst-case complexity of a sequential algorithm $\mathcal{A}$ for handling an instance $I_{\mathbb{P}}$ of $\mathbb{P}$ over G . For a parallel algorithm $\mathcal{A}_p$ for $\mathbb{P}$, we denote by $T_p(|I_{\mathbb{P}}|, |G|, n)$ its time taken for processing problem instance $I_{\mathbb{P}}$ on G using n processors. We say that algorithm $\mathcal{A}_p$ is *parallelly scalable relative to $\mathcal{A}$* if

$$T_p(|I_{\mathbb{P}}|, |G|, n) = O(T_s(|I_{\mathbb{P}}|, |G|)/n)$$

for any instance $I_{\mathbb{P}}$. That is, the parallel algorithm $\mathcal{A}_p$ is able to “linearly” reduce the sequential cost of a yardstick algorithm $\mathcal{A}$. Denote by SeqMiner the sequential version of RxGNNMiner that validates each RxGNN one by one, without multi-thread parallelism. We have the following.

Theorem 3: RxGNNMiner is parallelly scalable relative to sequential rule mining algorithm SeqMiner. $\square$

Proof. We show that the complexity of RxGNNMiner is $O(\frac{1}{n} \times |\mathcal{S}_{\mathcal{M}}|^K \sum_{X \in \text{pow}(\mathcal{P})} |G|^{|X|})$ with n processors, where K is the maximum number of motifs in $\mathcal{S}_{\mathcal{M}}$ that can compose a σ -frequent pattern Q (typically, $K \ll |\mathcal{S}_{\mathcal{M}}|$), $\mathcal{P}$ is the set of all potential predicates defined on a pattern Q and $\text{pow}(\mathcal{P})$ is the power set of $\mathcal{P}$. This is because (a) the computation is dominated by the validation of support and confidence, where the computational tasks are evenly distributed across n processors, and (b) it examines $\text{pow}(\mathcal{P})$ to generate preconditions X for each pattern Q at worst. $\square$

Space cost. The space complexity is dominated by the storage of minimal RxGNNs, which is $O(|\mathcal{S}_{\mathcal{M}}|^K \times (N + \#\max|\mathcal{P}|) + |G|)$, where N is the maximum number of vertices for each pattern Q and $\#\max = \lfloor \frac{|\mathcal{P}|}{2} \rfloor$ is the maximum number of minimal RxGNNs with the same Q .

Appendix A-4: Cover Computation

The problem of cover computation is as follows.

- **Input:** A set Σ of RxGNNs discovered.
- **Output:** A cover Σ^c of Σ such that (1) $\Sigma^c \models \Sigma$, i.e., $\Sigma^c \models \varphi$ for all $\varphi \in \Sigma$, and (2) $\Sigma^c \setminus \{\varphi\} \not\models \Sigma^c$ for any $\varphi \in \Sigma^c$.

Implication analysis (i.e., decide whether a rule φ is implied by a set Σ of rules, denoted as $\Sigma \models \varphi$) is embedded in cover computation. Below we first review the characterization of implication in [15] and adapt it to RxGNNs, which take the same classifier $\mathcal{M}$ in the consequences, followed by a parallel algorithm for cover computation along the same line as in [15].

Characterization. For $\varphi = Q[\bar{x}, x_0](X \rightarrow \neg\mathcal{M}(x_0))$, the set of RxGNNs embedded in Q , denoted by Σ_Q , consists of RxGNNs of the form $\varphi' = Q'[\bar{x}', x_0](f(X') \rightarrow \neg\mathcal{M}(x_0))$ such that (a) there exists an RxGNN $\varphi_m = Q'[\bar{x}', x_0](X' \rightarrow \neg\mathcal{M}(x_0))$ in Σ and (b) there exists an homomorphic mapping f from Q' to Q , where $f(X')$ is obtained by replacing x by $f(x)$ for every vertex x appearing in X' . Intuitively, Σ_Q consists of φ' in Σ s.t. Q' can be mapped to Q and hence, φ'

is enforced on every match of Q in a graph satisfying Σ .

For each Σ_Q and a given RxGNN $\varphi = Q[\bar{x}, x_0](X \rightarrow \neg\mathcal{M}(x_0))$, we define a set closure (Σ_Q, X) as follows:

- (a) all predicates in X are in closure (Σ_Q, X) ; and
- (b) for every $\varphi' = Q'[\bar{x}', x_0](X' \rightarrow \neg\mathcal{M}(x_0))$ in Σ_Q , $\neg\mathcal{M}(x_0)$ is in closure (Σ_Q, X) if either $X' = \emptyset$ or $X' \neq \emptyset$ and all predicates in X' can be deduced from closure (Σ_Q, X) via equality transitivity.

One can verify that, given Σ_Q , closure (Σ_Q, X) can be computed in PTIME, though determining Σ_Q itself is intractable. We say that closure (Σ_Q, X) is *conflicting* if there exists an attribute $x.A$ and distinct constants c and d such that both $x.A = c$ and $x.A = d$ can be derived.

The implication of RxGNNs is characterized as follows. For any set Σ of RxGNNs and $\varphi = Q[\bar{x}, x_0](X \rightarrow \neg\mathcal{M}(x_0))$, $\Sigma \models \varphi$ if and only if closure (Σ_Q, X) is conflicting or $\neg\mathcal{M}(x_0)$ is deducible from closure (Σ_Q, X) . We say that $\neg\mathcal{M}(x_0)$ is *deducible* from Σ and X if there exists a subset Σ_Q of RxGNNs of Σ that are embedded in Q and derived from Σ , such that $\neg\mathcal{M}(x_0)$ can be derived from closure (Σ_Q, X) via the transitivity of equality atoms. Intuitively, it says that either (a) Σ and (Q, X) are inconsistent and hence, $\neg\mathcal{M}(x_0)$ is their “logical consequence”; or (b) closure (Σ_Q, X) is consistent and $\neg\mathcal{M}(x_0)$ is enforced by Σ and (Q, X) .

Sequential cover computation. By utilizing the above characterization, a sequential cover computation algorithm works by inspecting RxGNNs in Σ one by one. For each $\varphi \in \Sigma$, it checks whether $\Sigma \setminus \{\varphi\} \models \varphi$ based on the characterization; if so, it removes φ from Σ , until no more φ can be removed and the resulting Σ is returned as Σ^c .

Following [15], one can verify that this algorithm correctly computes the cover Σ^c of Σ , since (a) each $\varphi \in \Sigma$ is checked based on the characterization of implication, and (b) upon termination, no φ can be further removed from Σ^c .

Parallelization. We parallelize the sequential algorithm. We partition the set Σ of RxGNNs into m groups $\Sigma_{Q_1}, \Sigma_{Q_2}, \dots, \Sigma_{Q_m}$, where each Σ_{Q_i} ($i \in [1, m]$) is the set Σ of RxGNNs that pertain to the same pattern Q_i . It suffices to check the implication of RxGNNs within each group, since the implication checking is *pairwise independent* among the groups [15].

For each Σ_{Q_i} , we create a work unit and collect it in a set W . The workload in W are evenly distributed to all workers. Upon receiving the assigned work unit W_j , each worker P_j invokes the sequential algorithm above to compute local cover Σ_j^c in parallel. When all work units are processed, the union of all Σ_j^c 's are returned.

Along the same lines as [15], one can readily verify that the parallel cover computation is also parallelly scalable.

Appendix B: Details about Local Counterfactual Explanation

This section provides additional details about our local counterfactual method CFE. We first present the details of DAG building and refinement in candidate space construction (Appendix B-1). We then show how to ensure the plausibility and feasibility of feature modifications (Appendix B-2).

Appendix B-1: DAG Building and Refinement

We present the details of DAG building and refinement.

DAG building. To build a DAG Q_D from Q , we treat the designated vertex x_0 as the root. We traverse Q in a BFS order from x_0 and direct all pattern edges from upper levels to lower levels. To process more selective vertices earlier, the directions between vertices at the same level are decided by an *infrequent-first* order: we group vertices by their labels (one group per label) and then sort the groups in the ascending order of their numbers of occurrence in G ; within each group, we sort vertices in the descending order of degrees.

Refinement. This step refines candidate space C_φ by propagating topological and logical constraints in both forward and backward directions. It implements dynamic programming between DAG Q_D and graph G [91].

More specifically, to refine $C_\varphi(x)$, it computes an auxiliary set $C'_\varphi(x)$ by dynamic programming such that vertex $v \in C'_\varphi(x)$ iff (a) $v \in C_\varphi(x)$, (b) there exists v_c adjacent to v such that $v_c \in C'_\varphi(x_c)$ for every child x_c of x in Q_D , and (c) there exists v_p such that $v.A \oplus v_p.B$ and $v_p \in C'_\varphi(x_p)$ for every variable predicate $x.A \oplus x_p.B$ in X .

Based on this, we compute $C_\varphi(x)$ by alternating forward refinement (propagating along Q_D) and backward refinement (along the reverse Q_D^{-1} of Q_D). This bidirectional refinement continues until no changes can be made to the candidate sets.

In practice, we can empirically set a maximum number of refinements, *e.g.*, when the filtering rate after the first three refinements is below 1% [91].

Appendix B-2: Feature Modification

To measure attribute importance, we adopt GINI values [85]; it shows how well X divides the pivots into different predicted classes of $\mathcal{M}$ [99], [100]. Denote the GINI value with (resp. without) $x.A$ in X by $\text{Gini}(X)$ (resp. $\text{Gini}(X_{\text{no}A})$). We define the importance of $x.A$ in X as $\Delta\text{Gini}_{x.A} = \text{Gini}(X_{\text{no}A}) - \text{Gini}(X)$, *i.e.*, the reduction of the GINI value by $x.A$. The larger the value, the more important $x.A$.

To ensure the plausibility and feasibility of feature modifications [67], we reuse the perplexity-based computation in Section IV-C, to leverage the capability and rich prior knowledge of LLMs. More specifically, for each attribute A , we pre-compute a function $\rho_A: \text{dom}(A) \times \text{dom}(A) \rightarrow \mathbb{R}^+$ for measuring the plausibility and feasibility of a modification, where $\text{dom}(A)$ is the domain of attribute A :

$$\rho_A(a, a') = \exp(-\log P_{\text{LLM}}(\text{nlp}(a \rightarrow a'))).$$

Here a (resp. a') is the original (resp. new) A -value, and $\text{nlp}(a \rightarrow a')$ is the natural language description of the value modification, denoted as $a \rightarrow a'$, *e.g.*, “the A -attribute value is modified from a to a' ”, and P_{LLM} is the probability estimated by LLMs. Intuitively, $\rho_A(a, a')$ assigns a lower perplexity score for more plausible and feasible modification.

In this paper, we only consider modification $a \rightarrow a'$ whose $\rho_A(a, a')$ is low enough *w.r.t.* a threshold.

Appendix C: Other Related Work

This section provides additional literature review. We first

present a detailed comparison of rule-based methods (Appendix C-1), and then review existing techniques that are incorporated as components of NEX (Appendix C-2).

Appendix C-1: Comparison of Rule-based Methods

Table V compares the expressivity of rule-based methods, *w.r.t.* their supported patterns, predicates, and consequences.

Rules are divided into two categories: rules for graph data and rules for tabular data. For each type of rules, we discuss (1) whether they support graph patterns (*e.g.*, general patterns or restricted patterns)? (2) What types of predicates the rules support, *e.g.*, constant predicates, variable predicates, inequality predicates ($\neq, \geq, \leq, >, <$), ML predicates, 1-WL predicates and negated predicates? (3) What consequences of the rules and the primary purposes of the rules are.

Rules for graph data. As shown in Table V, among all the prior graph rules, RxGNNs are the most expressive ones, by supporting both general graph patterns and all types of predicates for counterfactual explanation. The most related rules are REPs proposed in Makex [58], but they differ from RxGNNs in the following aspects.

- *Patterns.* RxGNNs employ general graph patterns while REPs are defined on restricted star patterns. General patterns are more expressive than star patterns, albeit at the cost of higher computational complexity, *e.g.*, it only takes PTIME to compute the matches of a given star pattern, but this problem becomes NP-hard for a general pattern.
- *Variable predicates.* RxGNNs support all types of predicates. In contrast, to support efficient validation of REPs, they only partially support variable predicates on the centers and leaves (*i.e.*, vertices without any child vertices) of the star patterns; moreover, each center/leaf can carry at most one variable predicate.
- *Purposes.* Although both RxGNNs and REPs are designed for GNN explanations, REPs are used to generate *factual explanations*, while RxGNNs aim to provide *counterfactual explanations*. The different purposes of REPs and RxGNNs call for different designs and methodologies. The former identifies key substructures or features as sufficient conditions supporting the prediction, without modifying the graph; this does not necessarily require negated predicates. In contrast, the latter requires suggesting appropriate modifications to the input graphs as necessary conditions; we need rules that support negated predicates.

To give concrete illustration about the expressivity for RxGNNs, consider $\varphi_1 = Q_1[\bar{x}, x_0](X_1 \rightarrow \neg \mathcal{M}_1(x_0))$ in Figure 1(e), where (1) Q_1 is a general pattern and (2) X_1 involves (a) constant predicate such as $x_0.\#AI_skill = \text{None}$, (b) variable predicate such as $x_2.name \neq x_3.name$, (c) comparison predicate $x_0.\#experience > 5$, and (d) negated 1-WL predicate $\neg 1WL(x_1)$. Note that φ_1 *cannot* be expressed by any existing rules, as elaborated below.

Prior rules REPs [58], TIEs [85] and GCRs [84] trade expressivity for efficiency by restricting graph patterns and variable predicates. REPs do not support general patterns and

Data	Rules	Patterns	Predicates						Consequences	Purpose of Rules
			Constant?	Variable?	beyond equality (e.g., $\neq$, $\geq$, $\leq$)?	ML?	1-WL?	ML/1-WL Negation?		
Graph	RxGNNs	general	✓	✓	✓	✓	✓	✓	$\neg \mathcal{M}(x_0)$	counterfactual explanation
	REPs [58]	star only	✓	partially	✓	✓	✓	×	$\mathcal{M}(x_0)$	factual explanation
	TIEs [85]	star only	✓	partially	✓	✓	×	×	(x, likes, y) (i.e., recommend item y to user x)	improve ML recommendation
	GARs [72]	general	✓	✓	✓	✓	×	×	all supported predicate types	association analyses
	GCRs [84]	star only	✓	partially	✓	✓	×	×	$x.A = y.B$ or $x.A = c$	graph data cleaning
	GFDs [71]	general	✓	✓	×	×	×	×	$x.A = y.B$ or $x.A = c$	graph data cleaning
	TACOs [83]	general	✓	✓	✓	✓	×	×	$((x, l, y), \tau)$ (i.e., the event specified by (x, l, y) will take place within time window τ)	association analyses for temporal event prediction
	GPARs [82]	general	×	×	×	×	×	×	(x, l, y) (i.e., the existence of edge (x, l, y))	association analyses
	GEDs [73]	general	✓	✓	×	×	×	×	$x.A = y.B$ or $x.A = c$	graph data cleaning
Tabular	[59], [60]	×	✓	×	✓	×	×	×	$\mathcal{M}(t) = i$	understand ML predictions
	RCs [61]	×	✓	×	✓	×	×	×	$\neg \mathcal{M}(t)$	counterfactual explanation
	REEs [101]	×	✓	✓	✓	✓	×	×	all supported predicate types	tabular data cleaning
	DCs [13]	×	✓	✓	✓	×	×	×	all supported predicate types	tabular data cleaning
	CFDs [102]	×	✓	✓	×	×	×	×	$t.A = s.B$ or $t.A = c$	conflict resolution
	MDs [103]	×	×	✓	×	✓	×	×	$t.id = s.id$	entity resolution
	FDs [104]	×	×	✓	×	×	×	×	$t.A = s.A$	database design

TABLE V: Expressivity comparison for different rules

negated predicates as noted earlier. GARs [72] and TACOs [83] partially alleviate this by supporting general patterns and richer predicates, but still omit an essential class: 1-WL predicates. As argued in [58], GNNs are at most as powerful as the 1-WL test, and most GNNs are based on it; hence, 1-WL is necessary to explain GNN behaviors. Without it, graph patterns with only simple arithmetic comparisons (e.g., $x.A \oplus y.B$, $x.A \oplus c$) lack sufficient expressive power for complex GNNs. GFDs [105] and GEDs [73] suffer the same limitation with even fewer predicates, while GPARs [82] are the most restrictive, defined solely by graph patterns without predicates.

Rules for tabular data. Rules are common in tabular data, supporting tasks such as ML explanation [61], [59], [60], data quality [101], [13], [102], [103], and database design [104]. Yet the structural complexity of graphs makes these tabular rules ill-suited for explaining GNNs, which rely on message passing over vertices and edges to capture rich patterns. In particular, rule-based explanation methods for tabular data [61], [59], [60] support only constant predicates, thus inheriting the same expressivity limitations as GFDs [105] and GEDs [73].

Appendix C-2: Reviews of Existing Techniques

This section reviews existing techniques that we leverage as components in NEX for building the first effective, scalable and unified framework for local and global GNN explanations.

Note that these techniques are not claimed as our major contributions and thus they are just briefly mentioned in the main text, with key adaptation highlighted. Interested readers can find full details in the cited papers.

Incremental discovery. Rule discovery aims to mine all rules whose supports and confidences exceed given thresholds σ and δ , respectively. Building on this, incremental rule discovery seeks to compute the updated rule set when thresholds change, i.e., $\sigma + \Delta\sigma$ or $\delta + \Delta\delta$. Algorithms are developed in [17] that minimize redundant recomputation, avoiding a full

restart of the costly discovery process. For support updates, it incrementally reuses the logic and anti-monotonicity property of a batch miner, while for confidence updates, it employs a sampling strategy to accelerate traversal of the search lattice.

Based on incremental discovery, the users can tune parameters σ and δ iteratively (Section IV-C). They may start with quick initial settings to obtain an initial rule set, then refine σ and δ using insights from the results. This allows mining high-quality rules first and then adjust σ and δ to find more [17]. Along the same lines, we tune motif size k as well.

Frequent pattern mining. Our motif identification summarizes key motifs from the set of subgraphs explaining GNN predictions. This step can naturally adopt frequent pattern mining techniques [10], [9], [12], [11]. We employ GASTON [9] to mine σ -frequent and k -bounded patterns from masked subgraphs for its efficiency. It discovers the frequent substructures by a layered growth scheme, i.e., a number of phases of increasing complexity. For example, it searches first for frequent paths, then frequent free trees and finally cyclic graphs.

Appendix D: Details about Experiments

This section provides additional experimental details. We first describe the datasets (Appendix D-1), then explain how baseline methods are adapted to our setting (Appendix D-2). **Finally, we show supplementary experiments (Appendix D-3).**

Appendix D-1: Additional Details about Datasets

In this section, we provide a detailed description of each dataset. We begin with a summary of four key statistical properties of four datasets in Table VI, followed by an elaboration on the vertices, edges, and attributes for each dataset.

Metrics. Four key statistical metrics were evaluated in Table VI, i.e., the average degree, the degree distribution entropy, the assortativity coefficient, and the spectral radius, all of which characterize the graph topology as elaborated below.

Statistic	Loan	Claim	Trans	Amazon
Average degree	4.00	4.76	1.65	3.91
Degree distribution entropy	1.01	1.29	1.24	1.28
Assortativity coefficient	-0.20	-0.18	-0.08	-0.12
Spectral radius	174.26	720.13	7.42	232.89

TABLE VI: Basic statistics of the datasets.

- *Average degree*. It measures how connected the vertices in a graph are on average. It is calculated by dividing the total number of edges by the total number of vertices.
- *Degree distribution entropy*. It measures the complexity of a graph's degree distribution. It is calculated as $-\sum_k P(k) \log_2 P(k)$, where $P(k)$ is the fraction of vertices with degree k . The higher the value is, the more diverse the degrees of the vertices are.
- *Assortativity coefficient*. It quantifies the tendency of vertices to connect with those of similar degrees, calculated by Pearson-correlation, in the range $[-1, 1]$, where a positive value indicates an *assortative* graph where vertices of similar degrees are connected, while a negative value represents a *disassortative* graph where the vertices of higher degree are connected with the ones of lower degree.
- *Spectral radius*. It measures the network connectivity. Specifically, it is defined as $\max_i |\lambda_i|$, where λ_i 's are the eigenvalues of the adjacency matrix $\mathbf{A}$ of a graph. The larger the value, the denser the graph.

Loan. The Loan dataset contains comprehensive information about loan applications and their associated risk factors, designed for loan default prediction and risk assessment tasks. The dataset captures multi-dimensional relationships between borrowers, their geographical locations, professional backgrounds, and loan characteristics through a heterogeneous graph structure. It enables advanced analysis of loan approval patterns, risk factors, and predictive modeling for financial institutions to make informed lending decisions.

Vertices. Loan has the following types of vertices.

- Loan vertices, which represent individual loan applications or loan products within the system.
- Applicant vertices, which represent loan applicants or borrowers seeking financial assistance.
- City vertices, which represent geographical city locations where applicants reside or loans are processed.
- State vertices, which represent state-level geographical divisions for broader regional analysis.
- Profession vertices, which represent occupational categories or job types of loan applicants.

Edges. Loan has the following types of edges.

- Applicant-Loan. It indicates which applicant has applied for or is associated with a specific loan.
- Applicant-City, the residential location of the applicant.
- City-State. It describes which state each city belongs to.
- Applicant-Profession, the occupational categories.

Attributes. The vertices of Loan carry the following attributes.

- VertexId: A unique identifier assigned to each vertex for individual tracking and reference across vertex types.
- Label: A categorical indicator used to distinguish vertex types (*i.e.*, Loan, Applicant, City, State or Profession).
- Income: The financial income level of the loan applicant, representing their earning capacity and financial stability.
- Age: The age of the loan applicant.
- Experience: The total work experience of the applicant.
- Married_Single: The marital status of the applicant.
- House_Ownership: The housing ownership status of the applicant (owned, rented, etc.), indicating the asset ownership.
- Car_Ownership: The ownership status of the applicant, representing additional asset ownership and financial capacity.
- Profession: The occupational category of the applicant.
- Current_Job_Year: The number of years the applicant has been in their current job position, for employment stability.
- Current_House_Year: The duration the applicant has been residing at their current address.
- Risk_Flag: A binary ground truth label indicating whether the loan application poses a high risk (1) or low risk (0).
- GNNPrediction: The GNN prediction for a specific loan.

Claim. The Claim dataset contains real-world medical insurance information designed for fraud detection in healthcare claims. It encompasses approximately 200,000 beneficiaries, over 5,000 healthcare providers, and around 550,000 medical insurance claims, with 38.1% of claims identified as fraudulent. The dataset features meticulous fraud labels and timestamps annotated by experts, providing typicality and authority for medical fraud detection research.

Vertices. Claim has the following types of vertices.

- Claim vertices, which represent individual medical insurance claims submitted for reimbursement.
- Beneficiary vertices, which represent patients or individuals who receive medical services and file insurance claims.
- Provider vertices, which represent healthcare providers, hospitals, or medical facilities that deliver services.
- DiagnosisGroup vertices, which represent categories of medical diagnoses or disease classifications.

Edges. Claim has the following types of edges.

- Beneficiary-Claim. It indicates which beneficiary has filed a specific medical insurance claim.
- Provider-Claim. It shows which healthcare provider is associated with or has submitted a particular claim.
- Claim-DiagnosisGroup. It connects claims to their corresponding medical diagnosis categories or disease groups.

Attributes. The vertices of Claim carry the following attributes.

- VertexId: A unique identifier assigned to each vertex for individual tracking and reference across all vertex types.
- Label: A categorical indicator used to distinguish vertex types (Claim, Provider, DiagnosisGroup or Beneficiary).
- ReimburseLevel: The level of the reimbursed value.
- DiagnosisCode: The medical diagnosis code assigned upon

hospital admission.

- DeductibleAmtPaid: The deductible amount paid by the beneficiary, for the out-of-pocket expense responsibility.
- Inpatient: A binary indicator for whether the medical service was provided as inpatient care or outpatient treatment.
- IsGroupFraud: A binary flag for whether the claim is part of a coordinated group fraud scheme with multiple parties.
- Gender: The gender of the beneficiary.
- Race: Racial demographic information of the beneficiary.
- RenalDiseaseIndicator: A binary indicator showing whether the beneficiary has renal (kidney) disease.
- State: The state information of the beneficiary/claim.
- County: The county information of the beneficiary/claim.
- numDiseases: The total number of diseases or medical conditions affecting the beneficiary.
- IsDead: A binary flag for whether the beneficiary is alive.
- IsFraud: A binary ground truth classification label indicating whether the claim is fraudulent (1) or legitimate (0).
- GNNPrediction: The GNN prediction for a specific claim.

Transaction. The Trans dataset contains comprehensive information about financial transactions for fraud detection. It comprises over 6.36 million transaction records, providing a rich and diverse collection of transactional data for analysis and modeling. Each transaction includes detailed information about the parties involved, monetary amounts, account balances, and fraud labels, making it particularly valuable for developing and evaluating fraud detection algorithms using graph-based machine learning approaches.

Vertices. Trans has the following types of vertices.

- Transaction vertices, which represent individual financial transactions within the system.
- Account vertices, which represent user accounts or entities participating in transactions.

Edges. Edges represent the relationships between Account vertices and Transaction vertices, indicating the flow of funds and transaction participation patterns within the financial network.

Attributes. The vertices of Trans carry the following attributes.

- VertexId: A unique identifier assigned to each vertex (transaction or account) for individual tracking and reference.
- Label: A categorical indicator used to distinguish vertex types (*i.e.*, Transaction or Account).
- Type: The category of financial transaction, including types such as PAYMENT, TRANSFER, CASH_OUT, DEBIT, etc., which help us classify different transaction behaviors.
- Amount: The monetary value involved in the transaction, for the financial magnitude and scale of each transaction.
- oldbalanceOrig: The account balance of the originating entity before the transaction occurs.
- newbalanceOrig: The updated account balance of the originating entity after the transaction is completed.
- oldbalanceDest: The account balance of the destination entity before receiving the transaction.
- newbalanceDest: The updated account balance of the des-

ination entity after receiving the transaction.

- isFraud: A binary ground truth classification label for whether the transaction is fraudulent (1) or legitimate (0).
- GNNPrediction: The GNN prediction for a transaction.

Amazon. The dataset captures an Amazon clothing e-commerce network, modeling interactions among users, products and reviews. It covers commercial relationships, user preferences, and product information, supporting analyses of purchasing patterns, review authenticity, and recommendations.

Vertices. Amazon has the following types of vertices.

- Product vertices, which represent individual clothing products available for purchase on the Amazon platform.
- Review vertices, which represent individual customer reviews and ratings submitted for products.
- User vertices, which represent customers who purchase products or submit reviews on the platform.

Edges. Amazon has the following types of edges.

- User-Review, the ownership between users and reviews.
- User-(buy)-Product, indicating a user has bought a product.
- User-(comment)-Product. It indicates that the user has commented on a product but not purchased yet.
- Product-Review. It connects products to the reviews they have received from customers.

Attributes. The vertices carry the following attributes.

- VertexId: A unique identifier assigned to each vertex for individual tracking and reference across all vertex types.
- Label: A categorical indicator used to distinguish between the three different vertex types (Product, Review or User).
- reviewText: The detailed content of the customer review.
- reviewSummary: A concise summary or title of the review.
- reviewScore: A numerical rating score given by the reviewer, typically on a scale representing satisfaction level.
- productTitle: The title of the product.
- brand: The manufacturer or brand name of the product.
- price: The selling price of the product.
- reviewTime: The timestamp for review submission.
- reviewerName: The identifier of the customer who submitted the review.
- style: The specific attributes of the clothing item such as color, size, material, fit, or design options.
- vote: The number of votes/likes received by a review, indicating its perceived usefulness by other users.
- description: The product description provided by the seller.
- category: A multi-class ground truth classification label indicating product categories, *i.e.*, men's clothing (0), women's clothing (1), women's underwear (2), outdoor sports (3), jewelry (4).
- GNNPrediction: The GNN prediction for a specific product.

Appendix D-2: Details about Baselines

This section presents more details about the baselines. Note that not all baselines are designed for vertex classification on

Perplexity-based predicate orderings (PPO)			Support-based predicate ordering (SO)		
	Example predicates	Semantic		Example predicates	Semantic
p_1	$x.\text{IsGroupFraud} = \text{true}$	A claim x is part of a coordinated group fraud scheme with multiple parties	q_1	$x.\text{DiagnosisCode} = 10000$	The medical diagnosis code assigned to a claim x is 10000
p_2	$x.\text{numDiseases} \geq c$	At least c diseases affect a beneficiary vertex x	q_2	$x.\text{Inpatient} = \text{false}$	A claim x is NOT provided with inpatient care
p_3	$x.\text{Inpatient} = \text{true}$	The claim x is provided with inpatient care	q_3	$x.\text{IsGroupFraud} = \text{false}$	The claim x is NOT part of a coordinated group fraud scheme with multiple parties
p_4	$\neg 1\text{WL}(x)$	The claim x is put in the negative class by 1-WL test	q_4	$x.\text{Race} = c$	The race id of the beneficiary x is c
p_5	$x.\text{ReimburseLevel} \geq c$	The reimbursed level of a claim x is at least c	q_5	$x.\text{Gender} = c$	The gender of the beneficiary x is c

TABLE VII: Case study of top-5 types of predicates in different orderings

LLM	Top-5 predicates returned				
	p_1	p_2	p_3	p_4	p_5
Ground truth (GT)	$x.\text{IsGroupFraud} = \text{true}$	$\neg 1\text{WL}(x)$	$x.\text{numDiseases} \geq c$	$x.\text{Inpatient} = \text{true}$	$x.\text{ReimburseLevel} \geq c$
Qwen2.5 (our default)	$x.\text{IsGroupFraud} = \text{true}$	$x.\text{numDiseases} \geq c$	$x.\text{Inpatient} = \text{true}$	$\neg 1\text{WL}(x)$	$x.\text{ReimburseLevel} \geq c$
Gemma3	$x.\text{numDiseases} \geq c$	$x.\text{IsGroupFraud} = \text{true}$	$x.\text{age} \geq c$	$\neg 1\text{WL}(x)$	$x.\text{ReimburseLevel} \geq c$
Phi3	$x.\text{Inpatient} = \text{true}$	$x.\text{IsGroupFraud} = \text{true}$	$x.\text{age} \geq c$	$x.\text{numDiseases} < c$	$x.\text{Inpatient} = \text{true}$
Llama3.2	$x.\text{numDiseases} \geq c$	$x.\text{IsGroupFraud} = \text{true}$	$x.\text{Inpatient} = \text{true}$	$x.\text{age} \geq c$	$\neg 1\text{WL}(x)$

TABLE VIII: Case study of top-5 types of predicates from different LLMs

heterogeneous graphs where vertices can carry both labels and attributes. For example, in our problem, even two vertices are both labeled as user vertices, they can still carry different attributes, *e.g.*, ages, names and genders; this may not be the case in other settings, *e.g.*, graph classification for molecules, where each vertices are atoms. Thus we will adapt the baselines for (a) vertex classification and (b) attributed graphs. Note that to tackle labels and attributes on vertices, existing GNN techniques typically associate vertices with feature vectors (in the form of numerical encodings, *e.g.*, word embedding or one-hot encoding) [48]. We follow this in our adaptations.

Local explanation. These methods aim to generate graph perturbations that flip GNN predictions. CF-Exp restricts itself to edge removals, whereas IndUCE and CFGraph allow both edge removals and insertions. CLEAR adopts a VAE framework, encoding an input graph into a latent space and decoding a counterfactual graph that preserves structural fidelity while flipping the prediction. GVEX, originally designed for graph classification, when adapted to vertex classification, it need to (a) construct a receptive field graph for each training vertex, and (b) then for each input graph, compute Jacobian matrices to identify vertices with maximal feature influence and performs graph bisection, gradually partitioning the graph into two subgraphs with opposing prediction outcomes.

Since most baselines ignore vertex feature modifications, we implemented a vertex-level masking scheme [86] under the same budget as edge modifications for fair comparison. In this scheme, baselines can apply feature masks to selected vertex feature vectors to perturb model predictions.

However, feature masking is often less intuitive than the explicit attribute dependencies in RxGNNs, where individual attributes are directly highlighted in the explanation. As noted in [58], applying the same feature mask to multiple vertices implicitly assumes that “features on different vertices are of equal importance”, which is typically not the case.

To optimize counterfactual methods, we apply graph perturbations incrementally, one at a time when feasible. Once the prediction flips, the process stops; otherwise, it continues until no further perturbations are possible.

In particular, we would like to express our sincere gratitude

to the authors of GVEX for their extensive discussions when we adapted their methods. Since GVEX is particularly designed for graph classification, it incurs performance degradation when adapting to vertex classification where the receptive field graph constructed for each training vertex (*i.e.*, the pivot vertex) is much larger than their original settings in [48]. This is mainly because GVEX requires identifying influential subgraphs, which incurs a much higher computational complexity than identifying edges to insert/remove like most other methods. Moreover, since GVEX requires computing the Jacobian matrix to measure feature influence, it has to compute the pairwise “influence” of all vertices in each receptive field graph (*i.e.*, $O(n^2)$ where n is the number of vertices in the graph constructed for a training vertex). Although we can approximate this by only considering the “influence” of all vertices to the pivot (*i.e.*, reduce from $O(n^2)$ to $O(n)$), GVEX is still slower than NEX by around two orders of magnitudes. In our experiments, we mainly adopted the original setting in [48] and followed the suggestions of the authors.

Global explanation. Both XGNN and GNNInt generate discriminative graph patterns to characterize model behavior: XGNN employs reinforcement learning to maximize specific predictions, while GNNInt uses a probabilistic generative approach with numerical optimization to learn explanation graph distributions. Both methods were originally developed for molecular graph analysis and focus exclusively on topological pattern generation without incorporating vertex features. GVEX summarizes global graph patterns from local explanation subgraphs. CFGraph generates global explanations by aggregation of multiple counterfactual instances, by analyzing the structural differences and edge modification patterns across instances to identify discriminative structures.

Since XGNN and GNNInt generate only topological patterns without vertex features, we adapt them by learning embeddings for each vertex type in these two methods. For GVEX and CFGraph, we adapt them by extracting vertex embeddings from the input graph and cluster them to obtain representative embeddings for each vertex type in these two methods.

Appendix D-3: Supplementary experiments

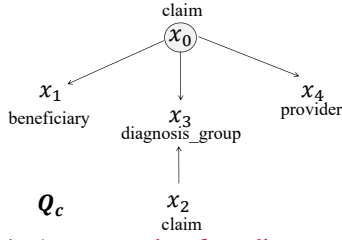


Fig. 11: A case study of predicate ordering

This section presents two case studies on Claim: (a) we first directly contrast the predicate orderings from PPO (perplexity-based ordering) vs. SO (support-based ordering) and then (b) we present the top-5 predicates returned by different LLMs. In addition, we provide an example prompt on Claim.

Predicate orderings from PPO and SO. Table VII reports the top-5 types of predicates in both PPO and SO, for pattern Q_c shown in Figure 11 on Claim, whose goal is to explain why the claim x_0 is predicated as fraudulent by a GNN model $\mathcal{M}$. For simplicity, predicates with similar semantic (e.g., x_0 .Inpatient = true and x_2 .Inpatient = true) are grouped into the same type.

As shown there, although the support-based processing ordering is popular in existing studies (e.g., [7]), predicates are ranked high because they are frequently applicable to a large portion of the data, rather than because they meaningfully contribute to explaining GNN predictions. This can lead to general predicates being prioritized in processing, rather than those that are truly discriminative, resulting in delays in pruning.

More specifically, consider q_1 in SO, which ensures that the medical diagnosis code assigned to a claim x is 10000. This predicate is ranked first because most of claims in the dataset are assigned this code. However, this does not indicate that claims with this code are more likely to be fraudulent. In fact, the fraud rate in claims using this code is among the lowest of all diagnosis codes. Similar issues apply to other top-ranked predicates, e.g., q_5 checks the gender of a beneficiary, which is a “general” checking that is able to cover enough instances to achieve high support, and q_3 is even contradictory to fraud prediction, as it requires a claim to be *not* part of a group fraud scheme. Nonetheless, due to the class imbalance in fraud prediction (i.e., the number of non-fraudulent claims significantly exceeds that of fraudulent ones), this predicate is ranked high.

In contrast, the predicates from PPO are more discriminative, as they describe essential factors for fraud detection, e.g., a claim is part of a group fraud scheme (p_1), involves inpatient care (p_3) and has a sufficiently high reimbursed value (p_5). These characteristics are commonly associated with fraudulent activities, representing typical tactics used to perpetrate fraud.

PPOs from different LLMs. We have tested the impact of LLMs on resulting predicate orderings, using Qwen2.5 [20], Gemma3 [21], Phi3 [23] and Llama3.2 [22] in Section VI and report a heatmap to visualize the orderings in Figure 10(c).

To fairly compare the orderings from different LLMs, we computed a ground truth (GT) order on predicates based on the

descending order of their frequencies appearing in final valid rules. Table VIII gives the concrete predicates from the GT order and different LLMs. Consistent with previous analysis, Qwen2.5 is the only LLM that captures all GT top-5 predicates and better reproduces the GT order than other LLMs, misordering only one predicate (p_4). Thus we used it as our default.

Example prompts for predicate orderings. To quantify the semantic correlation between predicates and GNN predictions, we compute the perplexity (PPL) of the predicate description given the graph context. Instead of prompting the LLM to generate a textual response, we leverage it to measure the perplexity of the target assertion, serving as a proxy for the causal strength between the attribute and the prediction result.

More specifically, we first assess the semantic correlation of the attribute itself (e.g., *isGroupFraud* is expected to more relevant than *Gender* in fraud detection). Then we evaluate the correlation direction (i.e., positive, negative, or non-linear), to selectively restrict the predicate search space (e.g., focusing on *isGroupFraud* = true, instead of *isGroupFraud* = false).

Below we show a condensed example prompt on Claim for attribute *isGroupFraud* in the core stage of attribute relevancy assessment. It serves as a template where specific attribute descriptions and their justifications (highlighted in *gray italics*) are replaceable to other attributes. The structural pattern is encoded directly within the textual context; the topological relationships to the pivot (e.g., a link between a *Provider* and the *Claim*) are articulated through natural language descriptions to ensure the LLM can capture the dependency effectively.

Codes for complete prompt generation for all data are in [1].

LLM Prompt: GNN Fraud Explainability (Condensed)

/ 1. Fixed domain knowledge to constrain and ground the LLM's interpretation. */*

Context: Healthcare Insurance Ecosystem. The insurance reimbursement system operates as a heterogeneous graph connecting claims, beneficiaries, providers, and diagnosis groups. Fraud detection aims to identify specific patterns, e.g., phantom billing, upcoding, or unbundling, that deviate from medical plausibility and contractual norms. Attributes are evaluated based on their forensic capacity to distinguish fraudulent intent from legitimate medical complexity.

/ 2. The specific attribute & predicate description. */*

Attribute & Predicate Description: ‘isGroupFraud’. *The attribute ‘isGroupFraud’ is a binary indicator sourced from law enforcement databases, flagging providers with documented criminal adjudications for organized billing conspiracies. It captures proven intent to defraud beyond reasonable doubt, distinct from administrative errors.*

/ 3. The hypothesis used for PPL calculation */*

Assertion & Forensic Reasoning. Criminological research on recidivism indicates prior adjudicated fraud is a robust predictor of future misconduct due to stable behavioral dispositions. Consequently, the presence of ‘isGroupFraud’ provides strong objective evidence of high-risk operational patterns, serving as a critical filter for identifying illicit behavior.